

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
**TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG**

---



**BÁO CÁO BÀI TẬP LỚN CƠ SỞ DỮ LIỆU**  
**ĐỀ TÀI: THIẾT KẾ CƠ SỞ DỮ LIỆU CHO HỆ THỐNG**  
**QUẢN LÝ RẠP CHIẾU PHIM**

**Giảng viên hướng dẫn: TS. Nguyễn Thị Oanh**

**Mã lớp: 166216**

**Sinh viên tham gia:**

**Phạm Minh Hùng – 202416928**

*Hà Nội, tháng 6, năm 2026*

# LỜI NÓI ĐẦU

Trong kỷ nguyên công nghệ số và sự phát triển mạnh mẽ của ngành công nghiệp giải trí, rạp chiếu phim không chỉ đơn thuần là nơi trình chiếu các tác phẩm điện ảnh mà đã vươn mình trở thành những tổ hợp dịch vụ phức hợp. Số lượng khách hàng gia tăng mỗi ngày kéo theo khối lượng dữ liệu khổng lồ cần được xử lý liên tục và chính xác: từ việc sắp xếp lịch chiếu, quản lý sơ đồ ghế ngồi, thông tin khách hàng, cho đến các dịch vụ đi kèm như đồ ăn, thức uống (F&B) và các chương trình tích điểm thành viên.

Thực tế vận hành cho thấy, việc quản lý thủ công hoặc sử dụng các nền tảng lưu trữ dữ liệu rời rạc, thiếu đồng bộ không còn đáp ứng được yêu cầu về tốc độ và tính an toàn. Những rủi ro phát sinh tại quầy vé như: bán trùng ghế, xuất vé quá giờ, sai sót trong áp dụng mã giảm giá, hay sự chậm trễ trong khâu tổng hợp báo cáo doanh thu có thể ảnh hưởng trực tiếp đến trải nghiệm của người xem và gây thất thoát tài chính cho doanh nghiệp.

Xuất phát từ những bài toán nghiệp vụ thực tiễn đó, nhóm chúng em đã thống nhất lựa chọn đề tài: **"Thiết kế Cơ sở dữ liệu cho hệ thống quản lý Rạp chiếu phim"**. Project này không chỉ dừng lại ở việc thiết kế và chuẩn hóa các bảng dữ liệu cơ bản (Phim, Phòng chiếu, Nhân viên, Hóa đơn,...), mà còn tập trung đi sâu vào việc tự động hóa các quy trình quản lý thông qua việc áp dụng các kỹ thuật Cơ sở dữ liệu nâng cao trên hệ quản trị PostgreSQL.

Trong quá trình thực hiện, nhóm đã nỗ lực vận dụng tối đa các kiến thức lý thuyết đã học để xây dựng một mô hình dữ liệu (ERD) tối ưu, đồng thời phát triển một ứng dụng Java cơ bản để mô phỏng sự tương tác thực tế với CSDL. Tuy nhiên, do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, đề tài chắc chắn không tránh khỏi những điểm thiếu sót. Nhóm chúng em rất mong nhận được những nhận xét, đánh giá và góp ý quý báu từ cô để có thể hoàn thiện hệ thống, mở ra hướng phát triển tích hợp thêm nền tảng đặt vé trực tuyến (Online Booking) trong tương lai.

# MỤC LỤC

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>LỜI NÓI ĐẦU.....</b>                                                   | <b>2</b>  |
| <b>MỤC LỤC .....</b>                                                      | <b>3</b>  |
| <b>I. THÔNG TIN CÁ NHÂN .....</b>                                         | <b>6</b>  |
| <b>II. ĐẶT VẤN ĐỀ .....</b>                                               | <b>7</b>  |
| 2.1. Bối cảnh thực tiễn của ngành giải trí và điện ảnh .....              | 7         |
| 2.2. Những thách thức trong quản lý rạp chiếu phim truyền thống .....     | 7         |
| 2.3. Sự cần thiết của việc xây dựng hệ thống quản lý hiện đại .....       | 8         |
| <b>III. GIỚI THIỆU HỆ THỐNG .....</b>                                     | <b>8</b>  |
| 3.1. Mục tiêu của hệ thống .....                                          | 8         |
| 3.2. Đối tượng sử dụng hệ thống .....                                     | 8         |
| 3.3. Các yêu cầu phi chức năng đối với CSDL .....                         | 9         |
| <b>IV. MÔ TẢ NGHIỆP VỤ THEO KỊCH BẢN CÓ THỂ XẢY RA ....</b>               | <b>9</b>  |
| 4.1. Kịch bản 01: Nghiệp vụ quản lý thành viên và tự động phân hạng ..... | 9         |
| 4.2. Kịch bản 02: Nghiệp vụ tổ chức và phân cấp nhân sự .....             | 11        |
| 4.3. Kịch bản 03: Nghiệp vụ lập lịch và điều phối suất chiếu .....        | 11        |
| 4.4. Kịch bản 04: Nghiệp vụ bán vé và giữ chỗ .....                       | 12        |
| 4.5. Kịch bản 05: Nghiệp vụ bán hàng và thanh toán .....                  | 12        |
| 4.6. Kịch bản 06: Nghiệp vụ quản lý nhập phim và thanh toán hóa đơn ..... | 13        |
| <b>V. MÔ TẢ CHỨC NĂNG .....</b>                                           | <b>14</b> |
| 5.1. Nhóm chức năng dành cho Quản lý (Admin/Manager) .....                | 14        |
| 5.2. Nhóm chức năng dành cho Nhân viên (Staff) .....                      | 14        |
| 5.3. Nhóm chức năng dành cho Khách hàng .....                             | 15        |
| 5.4. Nhóm chức năng hệ thống (System) .....                               | 15        |
| <b>VI. NỘI DUNG BÀI TẬP LỚN .....</b>                                     | <b>15</b> |
| 6.1. Phân tích mối quan hệ giữa các tập thực thể trước khi vẽ .....       | 15        |
| 6.1.1. Quản lý Khách hàng, Nhân viên và Tài khoản .....                   | 15        |
| 6.1.2. Quản lý Bán hàng và Giao dịch (Core Sales) .....                   | 16        |
| 6.1.3. Quản lý Vé và Suất chiếu .....                                     | 16        |
| 6.1.4. Quản lý Nhập phim và thanh toán hóa đơn nhập .....                 | 17        |



|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>VII. HƯỚNG DẪN CÀI ĐẶT &amp; SỬ DỤNG HỆ THỐNG QUẢN LÝ</b>              | <b>51</b> |
| 7.1. Yêu cầu cài đặt trước khi khởi chạy.....                             | 51        |
| 7.2. Cài đặt & cấu hình.....                                              | 51        |
| 7.3. Hướng dẫn sử dụng .....                                              | 55        |
| <b>VIII. ĐÁNH GIÁ HỆ THỐNG</b>                                            | <b>61</b> |
| 8.1. Ưu điểm.....                                                         | 61        |
| 8.1.1. Tính chuẩn hóa cao (Normalization đạt mức 3NF) .....               | 61        |
| 8.1.2. Phân chia Module logic chặt chẽ .....                              | 62        |
| 8.1.3. Bảo vệ tính toàn vẹn.....                                          | 62        |
| 8.2. Nhược điểm .....                                                     | 62        |
| 8.2.1. Chuẩn hóa quá mức gây áp lực truy vấn.....                         | 62        |
| 8.2.2. Thiếu nền tảng phát triển ứng dụng Mobile.....                     | 62        |
| 8.3. Những khó khăn trong quá trình phát triển hệ thống & giải pháp ..... | 63        |
| 8.3.1. Xung đột dữ liệu khi "Tranh cướp vé" .....                         | 63        |
| 8.3.2. Tắc nghẽn hiệu năng khi xuất báo cáo tổng hợp.....                 | 63        |
| 8.3.3. Kiểm soát logic độ tuổi và giá vé.....                             | 63        |

## I. Thông tin cá nhân & Nhiệm vụ

| MSSV      | Họ và tên      | Mail                                                                               |
|-----------|----------------|------------------------------------------------------------------------------------|
| 202416928 | Phạm Minh Hùng | <a href="mailto:Hung.PM2416928@sis.hust.edu.vn">Hung.PM2416928@sis.hust.edu.vn</a> |

| Họ và tên      | Nhiệm vụ                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phạm Minh Hùng | <ul style="list-style-type: none"><li>- Vẽ, chỉnh sửa sơ đồ thực thể - liên kết ERD, sơ đồ quan hệ</li><li>- Thêm, chỉnh sửa các tập thực thể, mối quan hệ giữa các tập thực thể</li><li>- Lấy dữ liệu, insert data vào database chuẩn bị viết queries</li><li>- Chuẩn bị queries &amp; phân chia</li><li>- Viết các queries, function, trigger, view, index</li><li>- Viết chương trình sinh dữ liệu random bằng Python</li><li>- Xây chương ứng dụng demo G09 Cinema bằng Java, kết nối với database</li><li>- Phát triển ứng dụng, debug lỗi phát sinh, tester, chỉnh sửa giao diện tương tác với user, thêm 1 số bộ phim lấy từ dữ liệu thực</li><li>- Hoàn thiện ứng dụng</li><li>- Chỉnh sửa hướng dẫn sử dụng, slide thuyết trình cho phù hợp với hệ thống</li><li>- Viết, chỉnh sửa báo cáo</li></ul> |

## II. Đặt vấn đề

### 2.1. Bối cảnh thực tiễn của ngành giải trí và điện ảnh

Trong những thập kỷ gần đây, điện ảnh đã trở thành một trong những lĩnh vực giải trí phổ biến và quan trọng nhất đối với đời sống tinh thần của con người. Cùng với sự phát triển không ngừng của công nghệ kỹ xảo và nội dung, nhu cầu đến rạp thưởng thức các tác phẩm điện ảnh trên màn hình lớn với âm thanh sống động của khán giả ngày càng tăng cao. Tại Việt Nam, các cụm rạp từ quy mô lớn (như CGV, Lotte Cinema, Galaxy Cinema, ...) đến các rạp địa phương đang mở rộng mạnh mẽ, bùng nổ về số lượng phòng chiếu và tần suất của các suất chiếu.

### 2.2. Những thách thức trong quản lý rạp chiếu phim truyền thống

Trước đây, việc quản lý rạp chiếu phim thường dựa vào các phương thức thủ công hoặc các hệ thống rời rạc. Điều này dẫn đến nhiều bất cập đáng kể:

- ❖ **Quản lý suất chiếu chồng chéo:** Việc sắp xếp lịch chiếu cho hàng chục bộ phim tại nhiều phòng máy khác nhau dễ xảy ra sai sót, trùng lặp khung giờ hoặc lãng phí tài nguyên phòng chiếu.
- ❖ **Khó khăn trong việc kiểm soát ghế ngồi:** Nhân viên khó có thể nắm bắt chính xác trạng thái ghế (ghế trống, ghế đã đặt, ghế hỏng) theo thời gian thực, dẫn đến tình trạng bán trùng ghế hoặc khách hàng không chọn được vị trí ưng ý.
- ❖ **Thất thoát doanh thu:** Quy trình bán vé và dịch vụ đi kèm (bắp nước) nếu không được số hóa sẽ rất khó kiểm soát chặt chẽ, dễ gây thất thoát tài chính và sai lệch số liệu kho vận.

- ❖ **Trải nghiệm khách hàng kém:** Khách hàng phải xếp hàng dài để mua vé, không thể đặt trước chỗ ngồi từ xa và khó khăn trong việc tra cứu thông tin lịch chiếu chính xác.

## 2.3. Sự cần thiết của việc xây dựng hệ thống quản lý hiện đại

Trong kỷ nguyên chuyển đổi số, việc ứng dụng Công nghệ thông tin để tối ưu hóa quy trình vận hành rạp chiếu phim là yêu cầu cấp thiết. Một hệ thống quản lý mạnh mẽ không chỉ giúp rạp vận hành trơn tru mà còn là nền tảng để triển khai các dịch vụ trực tuyến (Booking Online), chương trình khách hàng thân thiết và các chiến dịch Marketing cá nhân hóa.

Trái tim của hệ thống quản lý này chính là một **Cơ sở dữ liệu (Database)** được thiết kế khoa học, có khả năng lưu trữ lớn, đảm bảo tính nhất quán, an toàn dữ liệu và hỗ trợ truy xuất nhanh chóng. Do đó, đề tài "**Thiết kế Cơ sở dữ liệu cho Hệ thống Quản lý Rạp chiếu phim**" mang tính thực tiễn cao và là bước đi đầu tiên quan trọng để xây dựng một phần mềm quản lý hoàn chỉnh.

## III. Giới thiệu hệ thống

### 3.1. Mục tiêu của hệ thống

Hệ thống quản lý rạp chiếu phim được thiết kế nhằm mục đích công nghệ hóa toàn bộ hoạt động nghiệp vụ tại rạp, từ khâu quản lý thông tin phim, điều phối lịch chiếu, bán vé, quản lý dịch vụ ăn uống đến chăm sóc khách hàng và báo cáo doanh thu.

### 3.2. Đối tượng sử dụng hệ thống

1. **Người quản lý (Manager/Admin):** Có quyền cao nhất để quản lý thông tin phim, rạp, phòng chiếu, nhân viên và xem báo cáo tổng hợp.



2. **Nhân viên bán vé/Nhân viên quầy bắt nước:** Thực hiện các giao dịch trực tiếp với khách hàng tại quầy.
3. **Khách hàng:** Tra cứu lịch chiếu, thông tin phim và thực hiện đặt vé (nếu hệ thống mở rộng giao diện người dùng).

### 3.3. Các yêu cầu phi chức năng đối với CSDL

- **Tính nhất quán (Consistency):** Không xảy ra tình trạng bán quá số ghế hiện có.
- **Tính sẵn sàng (Availability):** Hỗ trợ truy xuất dữ liệu lịch chiếu và giá vé liên tục.
- **Tính bảo mật (Security):** Phân quyền rõ ràng giữa nhân viên và quản lý, bảo vệ thông tin cá nhân của khách hàng.
- **Khả năng mở rộng (Scalability):** Dễ dàng thêm rạp mới hoặc phim mới mà không làm thay đổi cấu trúc cốt lõi của hệ thống.

## IV. Mô tả nghiệp vụ theo kịch bản có thể xảy ra

### 4.1. Kịch bản 01: Nghiệp vụ quản lý thành viên và tự động phân hạng

- **Mô tả (Tình huống):** Khi một khách hàng đăng ký trở thành thành viên của rạp chiếu phim, hệ thống cần ghi nhận thông tin cá nhân và cấp tài khoản. Trong quá trình khách hàng sử dụng dịch vụ (mua vé, bắt nước), hệ thống sẽ theo dõi số điểm tích lũy và tự động điều chỉnh hạng thành viên khi đạt đủ điều kiện quy định.

- **Phân tích logic & Ràng buộc:**

- **Tính định danh và liên kết:** Mỗi khách hàng phải có một định danh duy nhất (CustomerID) liên kết chặt chẽ với một tài khoản đăng nhập (Account\_id). Mọi lịch sử giao dịch và quyền lợi đi kèm đều phải chiếu theo định danh này để đảm bảo tính cá nhân hóa và bảo mật thông tin.
- **Cơ chế ghi nhận điểm (Reward Points):** Điểm thưởng của thành viên là dữ liệu động, được tính toán và cập nhật (cộng dồn) ngay khi một giao dịch mua hàng được xác nhận thành công. Ràng buộc toàn vẹn yêu cầu điểm tích lũy phải luôn là số nguyên lớn hơn hoặc bằng 0.
- **Chuyển đổi trạng thái tự động:** Hạng thành viên (MembershipLevel) bị phụ thuộc hoàn toàn vào số Điểm tích lũy (RewardPoints). Khi điểm tích lũy đạt đến các mốc cấu hình sẵn (ví dụ: Đồng, Bạc, Vàng).
- **Quy định về Quản lý thành viên:** Hệ thống sẽ tự động cập nhật Hạng thành viên (MembershipLevel) của tài khoản khách hàng dựa trên số Điểm tích lũy (RewardPoints). Cụ thể:
  - ✓ **Từ 0 - 1999 điểm:** Hạng Đồng (Bronze)
  - ✓ **Từ 2000 - 4999 điểm:** Hạng Bạc (Silver)
  - ✓ **Từ 5000 - 9999 điểm trở lên:** Hạng Vàng (Gold)
  - ✓ **Từ 10000 điểm trở lên:** Hạng kim cương (Diamond). Khi khách hàng được cộng điểm sau mỗi lần mua vé và số điểm vượt qua cột mốc quy định, hệ thống sẽ tự động nâng hạng tương ứng."

## 4.2. Kịch bản 02: Nghiệp vụ tổ chức và phân cấp nhân sự

- **Mô tả (Tình huống):** Khi một nhân sự gia nhập hệ thống, hệ thống cần ghi nhận định danh và xác lập vị trí của họ trong sơ đồ tổ chức.
- **Phân tích logic & Ràng buộc:**
  - **Tính định danh:** Mỗi nhân viên phải có một định danh duy nhất để gắn trách nhiệm với mọi thao tác trên hệ thống sau này (như lập hóa đơn, tạo lịch chiếu).
  - **Mối quan hệ đệ quy (Self-referencing):** Nhân sự tồn tại mối quan hệ phân cấp nội bộ. Một nhân viên thông thường sẽ có một mối liên kết báo cáo trực tiếp tới một Quản lý (Manager).
  - **Phân quyền (Authorization):** Tùy thuộc vào vai trò được gán, quyền hạn thao tác dữ liệu sẽ bị giới hạn.

## 4.3. Kịch bản 03: Nghiệp vụ lập lịch và điều phối suất chiếu

- **Mô tả (Tình huống):** Quản lý rạp thao tác xếp phim "Lật Mặt" vào Phòng số 1 lúc 19:00. Hệ thống báo lỗi và từ chối lưu vì Phòng số 1 đang có lịch chiếu phim "Godzilla" từ 18:00 đến 20:00.
- **Phân tích logic & Ràng buộc:**
  - **Thực thể kết hợp:** Suất chiếu là dữ liệu duy nhất cấu thành từ: Mã phim + Mã phòng chiếu + Khung giờ.

- **Ràng buộc thời gian & không gian:** Validation nghiêm ngặt, tuyệt đối không cho phép hai suất chiếu giao thoa hoặc trùng lặp thời gian trong cùng một phòng chiếu.

#### 4.4. Kịch bản 04: Nghiệp vụ bán vé và giữ chỗ

- **Mô tả (Tình huống):** Khách tại quầy vừa click chọn ghế H1. Ngay lúc đó, một khách khác mở App điện thoại cũng chọn ghế H1 thì App báo lỗi "Ghế đang được giao dịch", ngăn chặn việc thanh toán.
- **Phân tích logic & Ràng buộc:**
  - **Cơ chế khóa ghế (Concurrency Control):** Ngay khi ghế được chọn (đang giao dịch) hoặc đã in vé, trạng thái ghế bị khóa trên toàn hệ thống để chống bán trùng (Double-booking).
  - **Tham chiếu dữ liệu:** Thực thể "Vé" bắt buộc chứa khóa ngoại trỏ đến đúng "Suất chiếu" và "Vị trí ghế".

#### 4.5. Kịch bản 05: Nghiệp vụ bán hàng và thanh toán

- **Mô tả (Tình huống):** Khách mua 2 vé phim và 1 Combo bắp nước chung trên một hóa đơn tổng 500k. Khách thanh toán bằng cách áp mã Voucher giảm 50k, đưa 200k tiền mặt và quét thẻ tín dụng 250k còn lại.
- **Phân tích logic & Ràng buộc:**
  - **Cấu trúc hóa đơn:** Một Hóa đơn (thực thể cha) có thể chứa nhiều Chi tiết hóa đơn (thực thể con) gồm cả vé và hàng hóa F&B.

- **Thanh toán linh hoạt:** Hệ thống cho phép chia nhỏ tổng tiền trên một hóa đơn để ghi nhận nhiều hình thức thanh toán (Tiền mặt, Thẻ, Voucher/Điểm) cùng lúc.

#### 4.6. Kịch bản 06: Nghiệp vụ quản lý nhập phim và thanh toán hóa đơn

- **Mô tả (Tình huống):** Hãng Warner Bros giao bản quyền phim "Dune 2" cho rạp. Quản lý rạp đăng nhập hệ thống, tạo và phê duyệt một Hóa đơn nhập (bản quyền phim) trị giá 500 triệu từ hãng này. Chỉ sau khi hóa đơn này được duyệt, phim "Dune 2" mới chính thức xuất hiện trong hệ thống để nhân viên bắt đầu xếp lịch chiếu.
- **Phân tích logic & Ràng buộc:**
  - **Nguồn gốc cung cấp:** Mọi hóa đơn nhập bản quyền bắt buộc phải gắn với một Nhà cung cấp duy nhất. Một nhà cung cấp có thể xuất nhiều hóa đơn cho rạp qua các đợt khác nhau.
  - **Khởi tạo dữ liệu phim:** Hóa đơn nhập là chứng từ đầu vào bắt buộc để đưa một bộ phim mới vào hệ thống khai thác. Không có hóa đơn nhập, phim không thể tồn tại hợp lệ trên hệ thống.
  - **Kiểm soát chi phí & Phân quyền:** Ràng buộc bảo mật chặt chẽ: Chỉ tài khoản được định danh là Quản lý mới có quyền tạo lập và phê duyệt các hóa đơn nhập này nhằm kiểm soát nguồn tiền và chi phí đầu vào của rạp.

## V. Mô tả chức năng

### 5.1. Nhóm chức năng dành cho Quản lý (Admin/Manager)

- **Quản lý danh mục:** Thêm, sửa, xóa thông tin phim (thể loại, thời lượng, mô tả), phòng chiếu, và thiết lập sơ đồ ghế ngồi.
- **Quản lý nhân sự:** Tạo tài khoản, phân quyền cho nhân viên và theo dõi hiệu suất làm việc (KPI) qua hóa đơn đã lập.
- **Quản lý nhập phim:** Quản lý thông tin nhà cung cấp và lập hóa đơn nhập phim/sản phẩm (Receiving Invoice).
- **Báo cáo & Thống kê:** Tổng hợp doanh thu theo ngày/tháng/năm, thống kê lượng vé bán ra, doanh thu F&B và báo cáo tồn kho sản phẩm.

### 5.2. Nhóm chức năng dành cho Nhân viên (Staff)

- **Bán vé:** Chọn phim, chọn suất chiếu, hiển thị sơ đồ ghế trống và thực hiện đặt chỗ cho khách.
- **Bán hàng (F&B):** Chọn các sản phẩm bắp, nước hoặc combo theo yêu cầu của khách hàng.
- **Thanh toán & in hóa đơn:** Tính tổng tiền, áp dụng mã giảm giá (nếu có), xử lý các phương thức thanh toán và in hóa đơn/vé cho khách.
- **Tra cứu thông tin & hỗ trợ dịch vụ:** Kiểm tra nhanh lịch chiếu và tình trạng ghế trống để tư vấn, hỗ trợ tích lũy điểm thưởng nâng hạng thành viên cho khách.

### 5.3. Nhóm chức năng dành cho Khách hàng

- **Mua vé Online:** Trực tiếp chọn suất chiếu, đặt ghế trống, thanh toán trực tuyến và nhận mã vé điện tử (QR Code).
- **Tra cứu:** Xem thông tin các phim đang chiếu và sắp chiếu; tra cứu lịch chiếu theo rạp hoặc theo phim và kiểm tra số điểm tích lũy/hạng thành viên.

### 5.4. Nhóm chức năng hệ thống (System)

- **Phân quyền:** Kiểm soát quyền truy cập dựa trên vai trò (Manager vs Staff).
- **Đồng bộ dữ liệu:** Đảm bảo cập nhật trạng thái ghế và kho hàng theo thời gian thực (Real-time).
- **Bảo mật:** Lưu trữ lịch sử đăng nhập và các thao tác nhạy cảm liên quan đến doanh thu và hóa đơn.

## VI. Nội dung bài tập lớn

### 6.1. Phân tích mối quan hệ giữa các tập thực thể trước khi vẽ

#### 6.1.1. Quản lý Khách hàng, Nhân viên và Tài khoản

- **Movie\_Account (1) - (1) Customer:**
  - Một khách hàng chỉ sở hữu một tài khoản thành viên duy nhất và một tài khoản chỉ định danh cho một khách hàng.
  - **Tầm quan trọng:** Đảm bảo tính duy nhất để quản lý điểm tích lũy, lịch sử xem phim và bảo mật thông tin cá nhân.

- **Manager (1) - (N) Staff:**

- Mỗi nhân viên (dù là nhân viên bán vé, nhân viên bắt nước hay nhân viên kỹ thuật) thường sẽ báo cáo trực tiếp và chịu sự điều hành của một người quản lý rạp.

### 6.1.2. Quản lý Bán hàng và Giao dịch (Core Sales)

- **Customer (1) - (N) Bill:**

- Một khách hàng có thể phát sinh nhiều hóa đơn (nhiều lần đi xem phim), nhưng một hóa đơn phải thuộc về một khách hàng cụ thể.

- **Staff (1) - (N) Bill:**

- Một nhân viên có thể lập nhiều hóa đơn cho khách.
- **Tầm quan trọng:** Giúp truy soát trách nhiệm và tính hiệu quả làm việc (KPI) của nhân viên bán vé/F&B.

- **Bill (N) - (N) Product:**

- Một hóa đơn có thể bao gồm nhiều sản phẩm (Bắp, nước, combo); ngược lại, một loại sản phẩm có thể xuất hiện trong nhiều hóa đơn khác nhau.
- **Tầm quan trọng:** Quản lý doanh thu dịch vụ đi kèm và kiểm soát kho (Inventory).

- **Bill (1) - (N) Payment:**

- Một hóa đơn có thể được thanh toán qua một hoặc nhiều đợt/phương thức (ví dụ: dùng voucher kết hợp tiền mặt).

### 6.1.3. Quản lý Vé và Suất chiếu

- **Ticket (N) - (1) MovieShow:**

- Một suất chiếu cụ thể có nhiều vé được bán ra, nhưng một vé chỉ thuộc về một suất chiếu mà thôi.



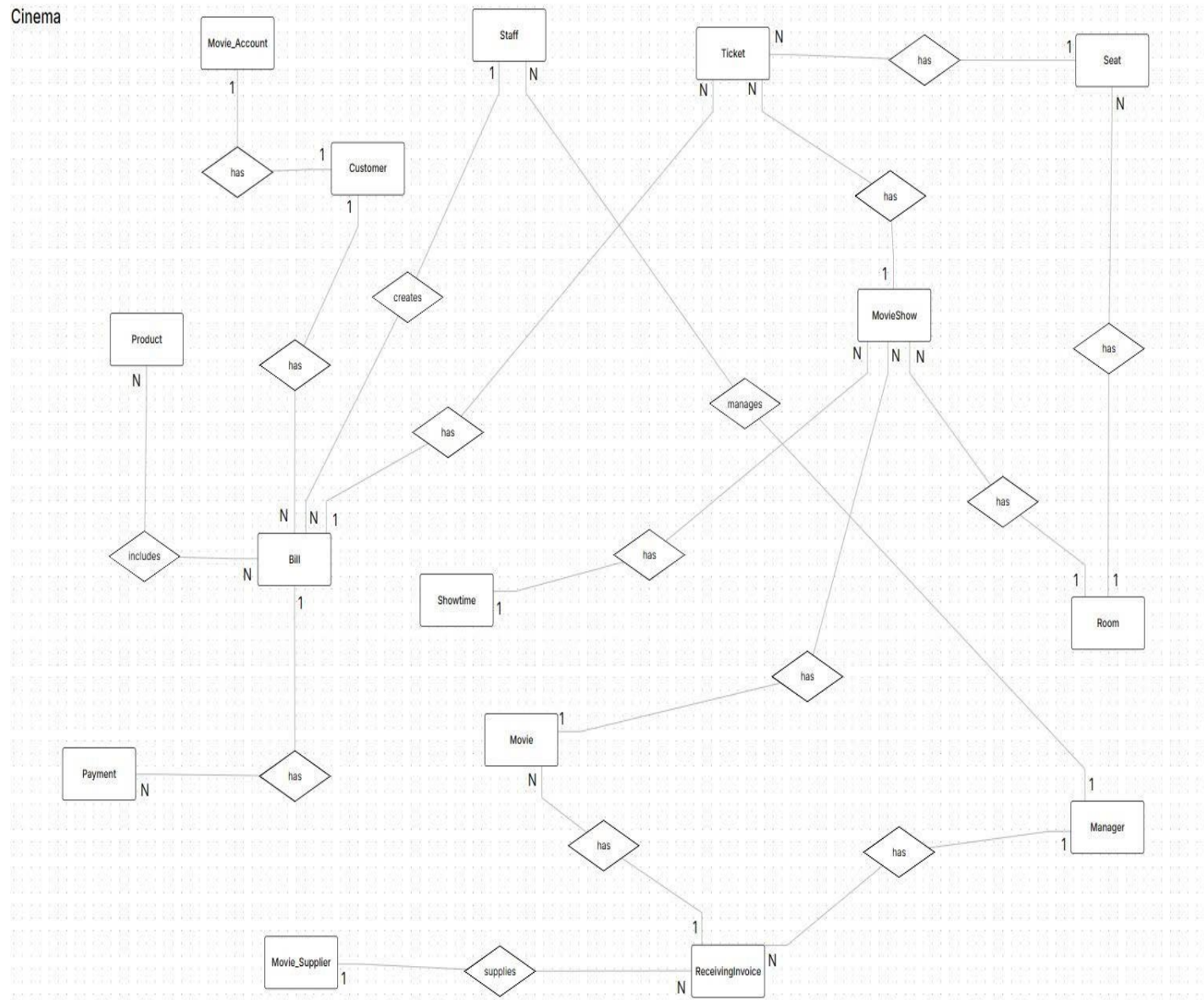
- **Room (1) - (N) Seat:**
  - Một phòng chiếu bao gồm nhiều ghế ngồi. Đây là mối quan hệ phân cấp không gian, giúp xác định vị trí chính xác của từng chiếc ghế thuộc phòng nào.
- **Ticket (N) - (1) Seat:**
  - Một vé chỉ tương ứng với một ghế ngồi và một ghế ngồi có thể được đặt ngồi bởi nhiều vé khác nhau.
  - **Tầm quan trọng:** Tránh tình trạng bán trùng ghế (Double-booking).
- **Ticket (N) - (1) Bill:**
  - Một **hóa đơn** có thể bao gồm nhiều **vé**. Ngược lại, mỗi chiếc **vé** chỉ có thể thuộc về duy nhất một **hóa đơn**.
- **MovieShow (N) - (1) Room / Showtime / Movie:**
  - Một phòng chiếu (Room) có thể tổ chức nhiều suất chiếu; một phim (Movie) cũng có thể chiếu nhiều suất; một khung giờ (Showtime) có thể áp dụng cho nhiều suất ở các phòng khác nhau.
  - **Tầm quan trọng:** Đây là "trái tim" của hệ thống, giúp tối ưu hóa lịch trình và công suất phòng máy.

#### 6.1.4. Quản lý Nhập phim và thanh toán hóa đơn nhập

- **Movie\_Supplier (1) - (N) ReceivingInvoice:**
  - Một nhà cung cấp phim có thể xuất nhiều hóa đơn nhập (bản quyền phim) cho rạp.
- **ReceivingInvoice (N) - (1) Movie:**
  - Hóa đơn nhập ghi nhận việc đưa các bộ phim mới vào hệ thống khai thác.
- **Manager (1) - (N) ReceivingInvoice:**
  - Chỉ có Quản lý mới có quyền tạo hoặc phê duyệt các hóa đơn nhập hàng/phim từ nhà cung cấp. Mối quan hệ này giúp kiểm soát chi phí đầu vào của rạp.

## 6.2. Sơ đồ thực thể - liên kết (ERD)

Link truy cập chi tiết ảnh: [Link\\_ERD](#)



## Phần mềm hỗ trợ: ERDPlus

**Quy ước ký hiệu trong sơ đồ:**

- Hình chữ nhật:** Đại diện cho các tập thực thể trong hệ thống.
- Hình thoi:** Minh họa các mối quan hệ (tập liên kết) giữa các thực thể.

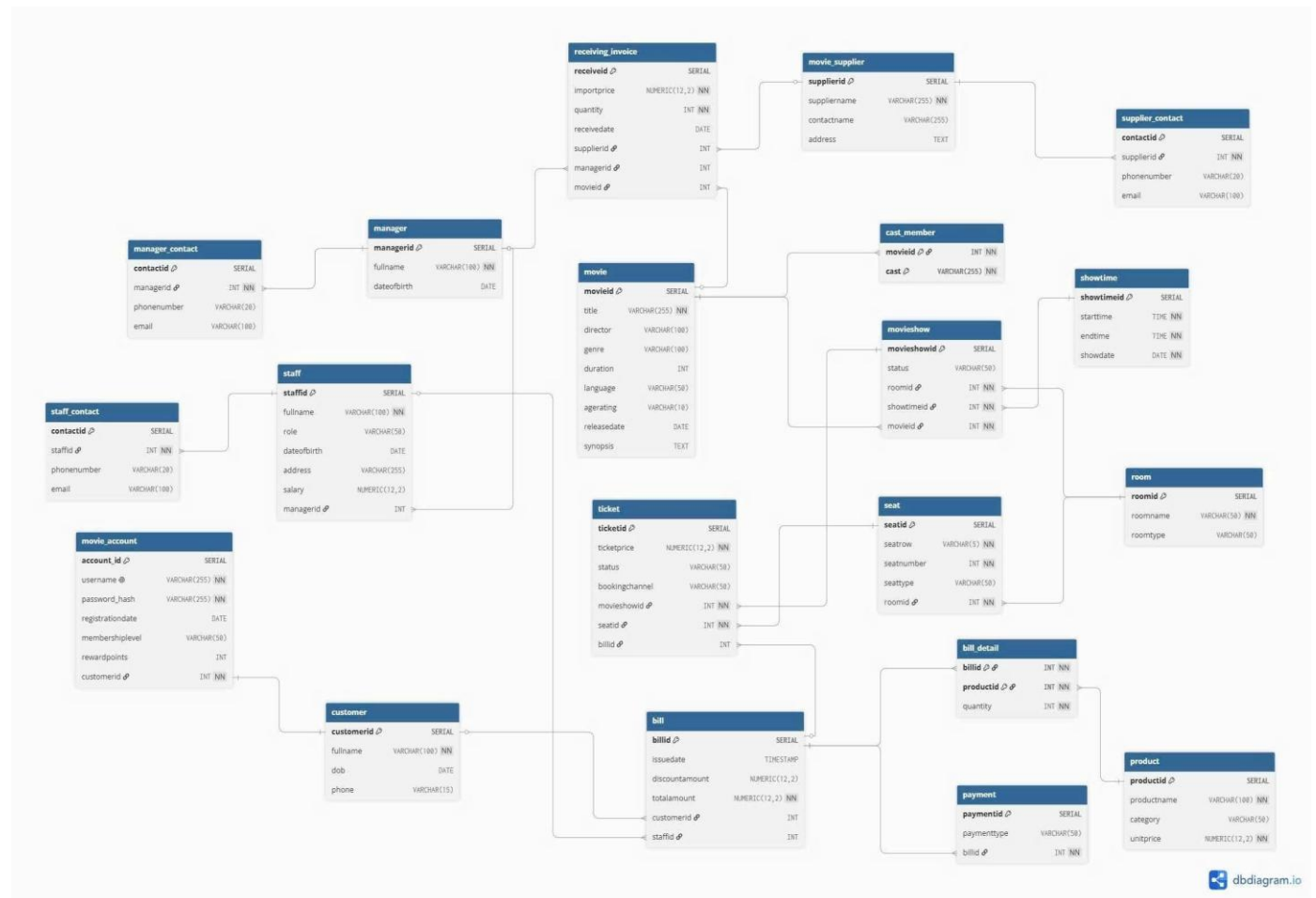
**Bảng chi tiết thuộc tính của các tập thực thể:**

| STT | Thực thể      | Thuộc tính                                                                                                                             |
|-----|---------------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Movie_Account | Account_id (PK), Username, Passwordhash, RegistrationDate, MembershipLevel (HangThanhVien), RewardPoints (DiemTichLuy)                 |
| 2   | Customer      | CustomerID (PK), Fullname                                                                                                              |
| 3   | Manager       | ManagerID (PK), FullName, PhoneNumber (đa trị), DateofBirth, Age (dẫn xuất), email (đa trị)                                            |
| 4   | Staff         | StaffID (PK), FullName, Role, PhoneNumber (đa trị), DateofBirth, Age (dẫn xuất), email (đa trị), Address (phức), Salary                |
| 5   | Bill          | BillID (PK), IssueDate, DiscountAmount, TotalAmount                                                                                    |
| 6   | Payment       | PaymentID (PK), PaymentType (e.g., Tiền mặt, Ví điện tử, Card)                                                                         |
| 7   | Ticket        | TicketID (PK), TicketPrice, Status (TrangThai - e.g., Sold, Reserved), BookingChannel (KenhDatVe - e.g., Counter, App, Website)        |
| 8   | Product       | ProductID (PK), ProductName (TenSanPham), Category (Loai - e.g., Popcorn, Drink, Combo), UnitPrice (DonGia)                            |
| 9   | Seat          | SeatID (PK), SeatRow (HangGhe - e.g., A, B, C), SeatNumber (SoThuTu - e.g., 1, 2, 3), SeatType (LoaiGhe - e.g., Standard, VIP, Couple) |
| 10  | Room          | RoomID (PK), RoomName (TenPhong),                                                                                                      |

|    |                  |                                                                                                                                                                                                                                    |
|----|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    |                  | RoomType (LoaiPhong - e.g., Standard, Gold Class, IMAX)                                                                                                                                                                            |
| 11 | Movie            | MovieID (PK), Title (TenPhim), Director (DaoDien), Cast (DienVien) (đã trị), Genre (TheLoai), Duration (ThoiLuong - minutes), Language (NgonNgu), AgeRating (DoTuoiQuyDinh), ReleaseDate (NgayKhoiChieu), Synopsis (TomTatNoiDung) |
| 12 | MovieShow        | MovieShowID (PK), Status (Available/Full/Finished)                                                                                                                                                                                 |
| 13 | Showtime         | ShowtimeID (PK), StartTime, EndTime, Date                                                                                                                                                                                          |
| 14 | Movie_Supplier   | SupplierID (PK), SupplierName, ContactName (Người liên hệ), PhoneNumber (đã trị), email (đã trị), Address (phức).                                                                                                                  |
| 15 | ReceivingInvoice | ReceiveID (PK), ReceiveDate, ImportPrice (Giá nhập), Quantity (Số lượng bản quyền/sản phẩm)                                                                                                                                        |

## 6.3. Sơ đồ Quan Hệ (Relational Schema)

Đường link dẫn vào xem chi tiết ảnh: [Link\\_Relational\\_Schema](#)



### 6.3.1. Nhóm Khách hàng & Tài khoản

- Customer (customerid, fullname, dob)
- Movie\_Account (account id, customerid, username, password\_hash, registrationdate, membershiplevel, rewardpoints)

### 6.3.2. Nhóm Nhân viên & Quản lý

- Manager (managerid, fullname, dateofbirth)
- Manager\_Contact (contactid, managerid, phonenumber, email)

- Staff (staffid, managerid, fullname, role, dateofbirth, address, salary)
- Staff\_Contact (contactid, staffid, phonenumber, email)

### 6.3.3. Nhóm Phim, Phòng chiếu & Suất chiếu

- Movie (movieid, title, director, genre, duration, language, agerating, releasedate, synopsis)
- Cast\_Member (movieid, cast)
- Showtime (showtimeid, starttime, endtime, showdate)
- Room (roomid, roomname, roomtype)
- Seat (seatid, roomid, seatrow, seatnumber, seattype)
- MovieShow (movieshowid, movieid, roomid, showtimeid, status)

### 6.3.4. Nhóm Hóa đơn & Vé

- Bill (billid, customerid, staffid, issuedate, discountamount, totalamount)
- Payment (paymentid, billid, paymenttype)
- Ticket (ticketid, movieshowid, seatid, billid, ticketprice, status, bookingchannel)

### 6.3.5. Nhóm Sản phẩm & Chi tiết bán hàng

- Product (productid, productname, category, unitprice)
- Bill\_Detail (billid, productid, quantity)

### 6.3.6. Nhóm Nhà cung cấp & Nhập hàng

- Movie\_Supplier (supplierid, suppliername, contactname, address)
- Supplier\_Contact (contactid, supplierid, phonenumber, email)

- ReceivingInvoice (receiveid, supplierid, managerid, movieid, receivedate, importprice, quantity)

## 6.4. Code SQL Minh Họa tạo bảng (DB\_Definition)

```
-- 1. NHÓM KHÁCH HÀNG & TÀI KHOẢN
CREATE TABLE Customer (
    customerid SERIAL PRIMARY KEY,
    fullname VARCHAR(255) NOT NULL,
    dob DATE
);
CREATE TABLE Movie_Account (
    account_id SERIAL PRIMARY KEY,
    customerid INT NOT NULL,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    registrationdate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    membershiplevel VARCHAR(50),
    rewardpoints INT DEFAULT 0,
    CONSTRAINT fk_account_customer FOREIGN KEY (customerid)
REFERENCES Customer(customerid)
);
-- 2. NHÓM NHÂN VIÊN & QUẢN LÝ
CREATE TABLE Manager (
    managerid SERIAL PRIMARY KEY,
    fullname VARCHAR(255) NOT NULL,
    dateofbirth DATE
);
CREATE TABLE Manager_Contact (
    contactid SERIAL PRIMARY KEY,
    managerid INT NOT NULL,
    phonenumber VARCHAR(15),
    email VARCHAR(100),
    CONSTRAINT fk_contact_manager FOREIGN KEY (managerid)
REFERENCES Manager(managerid)
);
CREATE TABLE Staff (
    staffid SERIAL PRIMARY KEY,
    managerid INT,
    fullname VARCHAR(255) NOT NULL,
```

```

        role VARCHAR(100),
        dateofbirth DATE,
        address VARCHAR(255),
        salary DECIMAL(15, 2),
        CONSTRAINT fk_staff_manager FOREIGN KEY (managerid)
REFERENCES Manager(managerid)
);
CREATE TABLE Staff_Contact (
    contactid SERIAL PRIMARY KEY,
    staffid INT NOT NULL,
    phonenumber VARCHAR(15),
    email VARCHAR(100),
    CONSTRAINT fk_contact_staff FOREIGN KEY (staffid) REFERENCES
Staff(staffid)
);
-- 3. NHÓM PHIM, PHÒNG CHIẾU & SUẤT CHIẾU
CREATE TABLE Movie (
    movieid SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    director VARCHAR(255),
    genre VARCHAR(100),
    duration INT,
    language VARCHAR(50),
    agerating VARCHAR(10),
    releasedate DATE,
    synopsis VARCHAR(1000)
);
CREATE TABLE Cast_Member (
    movieid INT,
    "cast" VARCHAR(255),
    PRIMARY KEY (movieid, "cast"),
    CONSTRAINT fk_movie FOREIGN KEY (movieid) REFERENCES
Movie(movieid) ON DELETE CASCADE
);
CREATE TABLE Showtime (
    showtimeid SERIAL PRIMARY KEY,
    starttime TIME NOT NULL,
    endtime TIME NOT NULL,
    showdate DATE NOT NULL
);
CREATE TABLE Room (
    roomid SERIAL PRIMARY KEY,

```



```

        roomname VARCHAR(50) NOT NULL,
        roomtype VARCHAR(50)
    );
CREATE TABLE Seat (
    seatid SERIAL PRIMARY KEY,
    roomid INT NOT NULL,
    seatrow CHAR(2) NOT NULL,
    seatnumber INT NOT NULL,
    seattype VARCHAR(50),
    CONSTRAINT fk_seat_room FOREIGN KEY (roomid) REFERENCES
Room(roomid)
);
CREATE TABLE MovieShow (
    movieshowid SERIAL PRIMARY KEY,
    movieid INT NOT NULL,
    roomid INT NOT NULL,
    showtimeid INT NOT NULL,
    status VARCHAR(50),
    CONSTRAINT fk_show_movie FOREIGN KEY (movieid) REFERENCES
Movie(movieid),
    CONSTRAINT fk_show_room FOREIGN KEY (roomid) REFERENCES
Room(roomid),
    CONSTRAINT fk_show_time FOREIGN KEY (showtimeid) REFERENCES
Showtime(showtimeid)
);
-- 4. NHÓM HÓA ĐƠN & VÉ
CREATE TABLE Bill (
    billid SERIAL PRIMARY KEY,
    customerid INT,
    staffid INT,
    issuedate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    discountamount DECIMAL(15, 2) DEFAULT 0,
    totalamount DECIMAL(15, 2) NOT NULL,
    CONSTRAINT fk_bill_customer FOREIGN KEY (customerid)
REFERENCES Customer(customerid),
    CONSTRAINT fk_bill_staff FOREIGN KEY (staffid) REFERENCES
Staff(staffid)
);
CREATE TABLE Payment (
    paymentid SERIAL PRIMARY KEY,
    billid INT NOT NULL,
    paymenttype VARCHAR(50),

```

```

        CONSTRAINT fk_payment_bill FOREIGN KEY (billid) REFERENCES
Bill(billid)
);
CREATE TABLE Ticket (
    ticketid SERIAL PRIMARY KEY,
    movieshowid INT NOT NULL,
    seatid INT NOT NULL,
    billid INT NOT NULL,
    ticketprice DECIMAL(15, 2) NOT NULL,
    status VARCHAR(50),
    bookingchannel VARCHAR(50),
    CONSTRAINT fk_ticket_show FOREIGN KEY (movieshowid)
REFERENCES MovieShow(movieshowid),
    CONSTRAINT fk_ticket_seat FOREIGN KEY (seatid) REFERENCES
Seat(seatid),
    CONSTRAINT fk_ticket_bill FOREIGN KEY (billid) REFERENCES
Bill(billid)
);
-- 5. NHÓM SẢN PHẨM & CHI TIẾT BÁN HÀNG
CREATE TABLE Product (
    productid SERIAL PRIMARY KEY,
    productname VARCHAR(255) NOT NULL,
    category VARCHAR(100),
    unitprice DECIMAL(15, 2) NOT NULL
);
CREATE TABLE Bill_Detail (
    billid INT NOT NULL,
    productid INT NOT NULL,
    quantity INT NOT NULL,
    PRIMARY KEY (billid, productid),
    CONSTRAINT fk_detail_bill FOREIGN KEY (billid) REFERENCES
Bill(billid),
    CONSTRAINT fk_detail_product FOREIGN KEY (productid)
REFERENCES Product(productid)
);
-- 6. NHÓM NHÀ CUNG CẤP & NHẬP HÀNG
CREATE TABLE Movie_Supplier (
    supplierid SERIAL PRIMARY KEY,
    suppliername VARCHAR(255) NOT NULL,
    contactname VARCHAR(255),
    address VARCHAR(255)
);

```

```

CREATE TABLE Supplier_Contact (
    contactid SERIAL PRIMARY KEY,
    supplierid INT NOT NULL,
    phonenumber VARCHAR(15),
    email VARCHAR(100),
    CONSTRAINT fk_contact_supplier FOREIGN KEY (supplierid)
REFERENCES Movie_Supplier(supplierid)
);
CREATE TABLE ReceivingInvoice (
    receiveid SERIAL PRIMARY KEY,
    supplierid INT,
    managerid INT,
    movieid INT,
    receivedate DATE DEFAULT CURRENT_DATE,
    importprice DECIMAL(15, 2) NOT NULL,
    quantity INT NOT NULL,
    CONSTRAINT fk_invoice_supplier FOREIGN KEY (supplierid)
REFERENCES Movie_Supplier(supplierid),
    CONSTRAINT fk_invoice_manager FOREIGN KEY (managerid)
REFERENCES Manager(managerid),
    CONSTRAINT fk_invoice_movie FOREIGN KEY (movieid) REFERENCES
Movie(movieid)
);

```

## 6.5. Insert dữ liệu cho database “G09\_Cinema” chuẩn bị viết queries

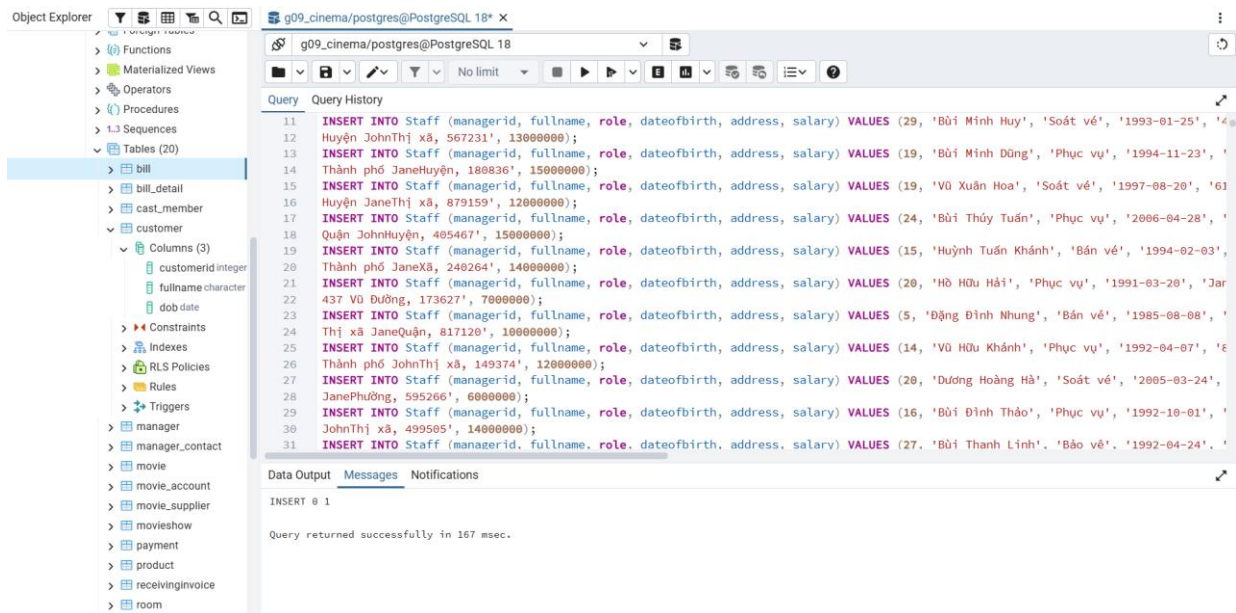
### 6.5.1. Tạo database

Tạo database tên “G09\_Cinema” qua câu lệnh “`create database G09_Cinema;`”

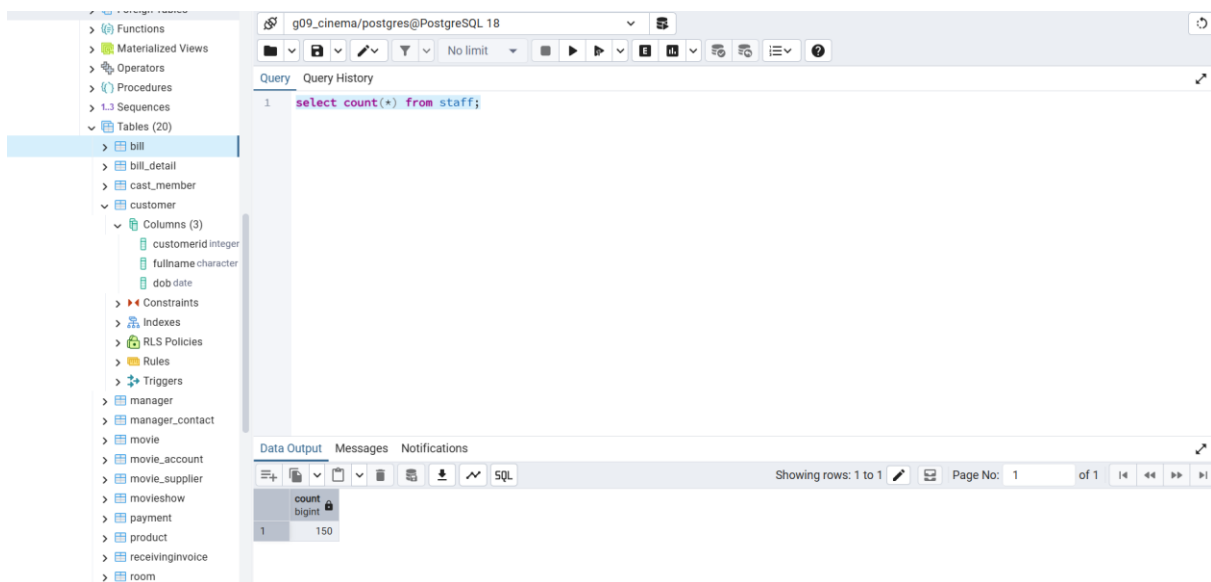
### 6.5.2. Insert dữ liệu

Để đảm bảo Cơ sở dữ liệu có đủ không gian mẫu phục vụ cho quá trình truy vấn, kiểm thử tính logic và đánh giá hiệu năng của hệ thống, thay vì nhập liệu thủ công, dữ liệu mẫu cần được nhóm sinh một cách tự động.

- **Công cụ và Kỹ thuật sử dụng:** Sử dụng ngôn ngữ lập trình **Python** kết hợp với thư viện **Faker**. Việc này giúp tạo ra tập dữ liệu giả lập (tên người, số điện thoại, địa chỉ, email...) hoàn toàn bằng tiếng Việt, đảm bảo tính thực tế và hợp lệ cho từng kiểu dữ liệu của CSDL. Sau khi tính toán, Python sẽ xuất ra một file script chứa các câu lệnh **“INSERT INTO (file DULIEU\_RAPPHIM.sql)”** để thực thi trực tiếp vào PostgreSQL.
- **Quy tắc toàn vẹn dữ liệu:** Quá trình sinh và chèn dữ liệu tuân thủ nghiêm ngặt thứ tự các ràng buộc khóa ngoại (Foreign Key). Các thực thể độc lập (Bảng cha: Customer, Manager, Movie, Room...) được khởi tạo và chèn trước. Các thực thể phụ thuộc (Bảng con: Staff, Ticket, Bill, MovieShow...) được chèn sau, với các khóa ngoại được chọn ngẫu nhiên từ dải ID hợp lệ của bảng cha.
- **Quy mô dữ liệu đã khởi tạo:**
  - Nhóm Khách hàng & Tài khoản: 300 Khách hàng (Customer), 300 Tài khoản (Movie\_Account).
  - Nhóm Nhân sự: 30 Quản lý (Manager), 150 Nhân viên (Staff).
  - Nhóm Phim & Lịch chiếu: 100 Bộ phim (Movie), 20 Phòng chiếu (Room), 200 Suất chiếu (MovieShow).
  - Nhóm Giao dịch: 300 Hóa đơn (Bill), 50 Sản phẩm bán/nước (Product), và hàng trăm vé xem phim (Ticket).
- **Trích đoạn mã SQL minh họa:**



## • Kiểm chứng:



-> Ảnh minh chứng trường hợp tượng trưng đã tạo đủ số lượng theo yêu cầu, chuẩn bị sẵn sàng cho viết các câu truy vấn, functionalities.

## 6.7. Các functionalities được cài đặt trong database

### 6.7.1. Views

#### 6.7.1.1. Tạo View Báo cáo doanh thu ngày (v\_Daily\_Revenue)

```
-- Sử dụng CREATE OR REPLACE để tự động ghi đè/cập nhật nếu View này đã từng được
tạo trước đó
CREATE OR REPLACE VIEW v_Daily_Revenue AS
SELECT
    -- Cột issuedate gốc là kiểu TIMESTAMP (ví dụ: '2024-05-10 14:30:00').
    -- Dùng cú pháp:: DATE để cắt bỏ phần giờ phút, ép về kiểu DATE (chỉ còn
    '2024-05-10') giúp việc gom nhóm chính xác hơn.
    issuedate::DATE AS ngay_giao_dich,

    -- Đếm tổng số lượng hóa đơn (mỗi billid là 1 hóa đơn) được bán ra trong ngày
    COUNT(billid) AS tong_so_hoa_don,

    -- Dùng hàm SUM cộng dồn toàn bộ cột totalamount để ra tổng số tiền thu được
    SUM(totalamount) AS tong_doanh_thu
FROM Bill
-- Lệnh GROUP BY bắt buộc phải có khi dùng COUNT/SUM. Nó sẽ gom tất cả các hóa
đơn có cùng chung 1 ngày lại thành 1 dòng để thống kê.
GROUP BY issuedate::DATE
-- Sắp xếp dữ liệu theo ngày giao dịch giảm dần (DESC), ngày nào mới nhất (hôm
nay) sẽ hiện lên trên cùng.
ORDER BY ngay_giao_dich DESC;
```

Tạo một VIEW có tên v\_Daily\_Revenue hiển thị ngày xuất hóa đơn (issuedate), tổng số hóa đơn được bán ra trong ngày đó, và tổng doanh thu thu được

-> Đơn giản hóa quá trình thống kê doanh thu

#### 6.7.1.2. Tạo View Thông tin Lịch chiếu (v\_Public\_Showtime)

```
CREATE OR REPLACE VIEW v_Public_Showtime AS
SELECT
    m.title AS ten_phim,
    r.roomname AS phong_chieu,
    s.showdate AS ngay_chieu,
    s.starttime AS gio_bat_dau
FROM MovieShow ms
```

```
-- Bảng MovieShow (Bảng trung tâm) chỉ chứa các con số ID. Ta phải dùng lệnh JOIN
nối với 3 bảng cha để lấy ra chữ/thông tin thực tế.
JOIN Movie m ON ms.movieid = m.movieid      -- Nối với bảng Movie để bốc
được tên phim (title)
JOIN Room r ON ms.roomid = r.roomid          -- Nối với bảng Room để bốc được
tên phòng (roomname)
JOIN Showtime s ON ms.showtimeid = s.showtimeid -- Nối với bảng Showtime để bốc
được ngày và giờ chiếu
-- Sắp xếp ưu tiên: Ngày chiếu tăng dần (ASC - Ngày sớm hiện trước), sau đó đến
Giờ chiếu tăng dần (ASC - Ca sáng hiện trước ca chiều).
ORDER BY s.showdate ASC, s.starttime ASC;
```

Tạo một VIEW hiển thị Lịch chiếu phim cho khách hàng xem trên web

(chỉ gồm title, roomname, starttime, showdate).

View này cung cấp đầy đủ thông tin lịch chiếu cho người dùng cuối (khách hàng xem trên Web/App) nhưng che giấu hoàn toàn các trường dữ liệu nhạy cảm, giúp chống lại các rủi ro khai thác dữ liệu nội bộ

-> **Đóng vai trò như một lớp bảo vệ cấu trúc cơ sở dữ liệu**

## 6.7.2. Index

### 6.7.2.1. Tạo index cho Customer (idx\_customer\_fullname)

```
CREATE INDEX idx_customer_fullname
ON Customer (fullname);
```

Giải quyết bài toán mở rộng (Scalability) của hệ thống. Index tự động tạo ra một cấu trúc tìm kiếm ngầm (B-Tree), giúp các thao tác tra cứu tên khách hàng đạt tốc độ gần như lập tức (Real-time). Kỹ thuật này giúp hệ thống tránh được hiện tượng thất cổ chai do phải quét toàn bộ bảng (Full Table Scan) khi lượng khách hàng tăng lên hàng chục ngàn người trong thực tế

### 6.7.2.2. Tạo index cho Movie (idx\_movie\_search)

```
CREATE INDEX idx_movie_search
ON Movie (title, genre);
```

Chức năng tra cứu phim theo tên gọi hoặc lọc theo thể loại là tính năng cốt lõi, có lưu lượng truy cập (traffic) cao nhất trên giao diện ứng dụng đặt vé của khách

hàng. Việc tạo Composite Index (chỉ mục kết hợp) trên hai cột title và genre giúp cơ sở dữ liệu tối ưu hóa các câu lệnh truy vấn có điều kiện WHERE phức tạp.

#### 6.7.2.3. Tối ưu hóa các lệnh JOIN trung tâm (B-Tree Index trên Khóa ngoại)

```
CREATE INDEX IF NOT EXISTS idx_movieshow_movie_showtime  
ON MovieShow (movieid, showtimeid);
```

**Chỉ mục trên Khóa ngoại:** Bảng MovieShow là bảng trung gian liên kết đa chiều. Việc tạo idx\_movieshow\_movie\_showtime giúp giảm thiểu chi phí quét toàn bảng (Full Table Scan) khi hệ thống thực hiện các phép JOIN dày đặc để xuất lịch chiếu lên giao diện người dùng.

#### 6.7.2.4. Tối ưu hóa Báo cáo Thống kê & Doanh thu

```
CREATE INDEX IF NOT EXISTS idx_bill_issuedate  
ON Bill (issuedate DESC);
```

**Chỉ mục tối ưu Sắp xếp:** Cột issuedate của bảng Bill được gắn index theo chiều giảm dần (DESC). Điều này giúp các View báo cáo và hàm (Function) thống kê doanh thu hoạt động trơn tru khi truy xuất dữ liệu của những ngày gần nhất.

#### 6.7.2.5. Tối ưu hóa sơ đồ ghế và chống bán trùng

```
CREATE INDEX IF NOT EXISTS idx_ticket_seat_check  
ON Ticket (movieshowid, seatid);
```

**Chỉ mục Phức hợp:** Index idx\_ticket\_seat\_check trên cặp khóa (movieshowid, seatid) của bảng Ticket đóng vai trò cốt lõi trong việc tải sơ đồ ghế (Seat Map) lên App theo thời gian thực (Real-time), đồng thời tăng tốc độ kiểm tra ràng buộc cho Trigger chống Double-booking.

#### 6.7.2.6. Bật tính năng tìm kiếm Full-Text Search

```
-- Khách gõ "lat mat" hay "LẬT mặt" thì hệ thống vẫn dịch ra vector từ khóa tiếng  
Việt để tìm đúng phim.  
CREATE INDEX IF NOT EXISTS idx_movie_title_fts  
ON Movie USING gin(to_tsvector('simple', title));
```



**Chỉ mục Full-Text Search:** Nâng cấp công cụ tìm kiếm phim bằng việc áp dụng kiến trúc Inverted Index (GIN) và chuyển đổi chuỗi văn bản sang tsvector. Kỹ thuật này giúp ứng dụng xử lý linh hoạt các truy vấn tìm kiếm tự do của khách hàng trên thanh Search mà các toán tử LIKE thông thường không thể tối ưu.

## 6.7.3. Function

### 6.7.3.1. Tính tuổi khách hàng

```
CREATE OR REPLACE FUNCTION fn_CalculateAge(p_customerid INT)
RETURNS INT AS $$
DECLARE
    v_age INT; -- Khai báo biến cục bộ để lưu trữ kết quả tuổi tính được
BEGIN
    -- Sử dụng hàm AGE() để tính khoảng thời gian từ ngày sinh (dob) đến hiện tại
    (CURRENT_DATE)
    -- Hàm EXTRACT(YEAR FROM ...) giúp "bóc" lấy đúng con số năm (tuổi) từ khoảng
    thời gian đó
    -- Từ khóa INTO dùng để gán kết quả vừa truy vấn được vào biến v_age
    SELECT EXTRACT(YEAR FROM AGE(CURRENT_DATE, dob)) INTO v_age
    FROM Customer
    WHERE customerid = p_customerid;

    -- Trả kết quả cuối cùng về cho người gọi hàm
    RETURN v_age;
END;
$$ LANGUAGE plpgsql;
```

- **Đảm bảo tính chính xác tuyệt đối:** Khắc phục nhược điểm của việc tạo cột "Tuổi" tĩnh (dễ bị sai lệch qua từng năm). Hàm tính toán realtime dựa trên ngày sinh (dob) giúp dữ liệu tuổi luôn được cập nhật đúng với thời điểm hiện tại.
- **Nền tảng kiểm duyệt nghiệp vụ:** Là công cụ lõi giúp hệ thống (hoặc thu ngân) đối chiếu và tự động từ chối bán vé nếu khách hàng chưa đủ tuổi xem các bộ phim có dán nhãn giới hạn (C16, C18).

### 6.7.3.2. Thống kê doanh thu phim

```
CREATE OR REPLACE FUNCTION fn_CalculateMovieRevenue(p_movieid INT)
```

```

RETURNS NUMERIC AS $$
DECLARE
    v_total_revenue NUMERIC; -- Khai báo biến lưu trữ tổng doanh thu (kiểu số thực)
BEGIN
    -- Truy vấn để tính tổng tiền
    SELECT COALESCE(SUM(t.price), 0) INTO v_total_revenue
    FROM Ticket t
    -- Liên kết vé với suất chiếu để biết vé này thuộc về suất chiếu nào
    JOIN MovieShow ms ON t.movieshowid = ms.movieshowid
    -- Điều kiện lọc: Chỉ lấy những suất chiếu của đúng bộ phim đang được truyền vào hàm
    WHERE ms.movieid = p_movieid;

    -- Trả kết quả doanh thu về
    RETURN v_total_revenue;
END;
$$ LANGUAGE plpgsql;

```

- **Che giấu sự phức tạp:** Gom toàn bộ logic tính toán, kết nối (JOIN) phức tạp giữa các bảng thành một lệnh gọi hàm duy nhất. Giúp lập trình viên phía giao diện (Java) truy xuất số liệu dễ dàng mà không cần viết lại SQL.
- **Tối ưu hóa module Báo cáo:** Hỗ trợ cho Quản lý rạp trong việc đo lường hiệu quả kinh doanh, có thể tái sử dụng hàm này ở bất kỳ đâu trong hệ thống để xuất báo cáo doanh thu độc lập cho từng rạp phim một cách tức thời.

## 6.7.4. Trigger

### 6.7.4.1. Chống bán trùng ghế

```

-- Bước 1: Viết logic kiểm tra trùng lặp
CREATE OR REPLACE FUNCTION fn_PreventDoubleBooking()
RETURNS TRIGGER AS $$
BEGIN
    -- Kiểm tra xem đã có dòng nào trong bảng Ticket trùng khớp cả mã suất chiếu và mã ghế chưa
    IF EXISTS (
        SELECT 1 FROM Ticket
        WHERE movieshowid = NEW.movieshowid AND seatid = NEW.seatid
    ) THEN
        -- Bắt lỗi và tự động Rollback (hủy) giao dịch
        RAISE EXCEPTION 'Lỗi giao dịch: Ghế này đã có người đặt!';
    END IF;
END;

```

```

        END IF;
        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Bước 2: Gắn Trigger vào bảng Ticket
CREATE TRIGGER trg_Before_Insert_Ticket_DoubleBooking
BEFORE INSERT ON Ticket
FOR EACH ROW
EXECUTE FUNCTION fn_PreventDoubleBooking();

```

**Đánh giá nghiệp vụ:** Đảm bảo tính nhất quán và toàn vẹn dữ liệu. Xử lý triệt để bài toán tranh chấp tài nguyên (Concurrency Control) khi có nhiều thu ngân hoặc người dùng trên App cùng thao tác đặt vé vào một thời điểm, ngăn ngừa sự cố trùng ghế.

#### 6.7.4.2. Kiểm soát giảm giá

```

CREATE OR REPLACE FUNCTION fn_DiscountAudit()
RETURNS TRIGGER AS $$
BEGIN
    -- Kiểm tra nếu số tiền giảm giá lớn hơn hoặc bằng tổng tiền (sau khi đã khấu trừ)
    -- Sử dụng COALESCE để đề phòng trường hợp discountamount bị bỏ trống (NULL)
    IF COALESCE(NEW.discountamount, 0) >= NEW.totalamount THEN
        RAISE EXCEPTION 'Lỗi nghiệp vụ: Số tiền giảm giá không được lớn hơn hoặc bằng tổng tiền thanh toán!';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_Before_Insert_Update_Bill_Discount
BEFORE INSERT OR UPDATE ON Bill
FOR EACH ROW
EXECUTE FUNCTION fn_DiscountAudit();

```

**Đánh giá nghiệp vụ:** Hoạt động như một lớp chốt chặn an ninh tài chính. Giúp kiểm soát rủi ro, ngăn chặn các sai sót nhập liệu (Human Error) hoặc các hành vi gian lận trục lợi nội bộ từ các chương trình khuyến mãi của hệ thống.

### 6.7.4.3. Tự động nâng hạng thành viên

```
CREATE OR REPLACE FUNCTION fn_UpgradeMembership()
RETURNS TRIGGER AS $$
BEGIN
    -- Logic phân cấp hạng thành viên dựa trên số điểm hiện có
    IF NEW.rewardpoints >= 10000 THEN
        NEW.membershiplevel := 'Kim Cương';
    ELSIF NEW.rewardpoints >= 5000 THEN
        NEW.membershiplevel := 'Vàng';
    ELSIF NEW.rewardpoints >= 2000 THEN
        NEW.membershiplevel := 'Bạc';
    ELSE
        NEW.membershiplevel := 'Đồng';
    END IF;
    -- Biến NEW tự động ghi nhận giá trị mới trước khi lưu vào DB, không cần lệnh
    UPDATE
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Bắt sự kiện UPDATE xảy ra TRÊN CỘT rewardpoints của bảng Movie_Account
CREATE TRIGGER trg_Before_Update_Account_Rank
BEFORE UPDATE OF rewardpoints ON Movie_Account
FOR EACH ROW
EXECUTE FUNCTION fn_UpgradeMembership();
```

**Đánh giá nghiệp vụ:** Đây logic xét duyệt thứ hạng từ tầng Ứng dụng (App/Web) xuống trực tiếp tầng Cơ sở dữ liệu. Việc này giúp giảm tải cho Server xử lý, đồng thời kích thích nhu cầu tiêu dùng của khách hàng khi họ thấy thứ hạng được cập nhật ngay lập tức.

### 6.7.4.4. Chống bán vé vượt quá giờ

```
CREATE OR REPLACE FUNCTION fn_PreventOverdueTicket()
RETURNS TRIGGER AS $$
DECLARE
    v_show_timestamp TIMESTAMP;
BEGIN
    -- Lấy thông tin ngày và giờ của suất chiếu, gộp lại thành 1 chuỗi thời gian
    cụ thể
    SELECT (s.showdate + s.starttime) INTO v_show_timestamp
    FROM MovieShow ms
    JOIN Showtime s ON ms.showtimeid = s.showtimeid
```

```

WHERE ms.movieshowid = NEW.movieshowid;

-- So sánh với thời gian thực tế của hệ thống máy chủ
IF v_show_timestamp < CURRENT_TIMESTAMP THEN
    RAISE EXCEPTION 'Giao dịch từ chối: Không thể xuất vé, suất chiếu này đã
bắt đầu hoặc đã kết thúc!';
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_Before_Insert_Ticket_Overdue
BEFORE INSERT ON Ticket
FOR EACH ROW
EXECUTE FUNCTION fn_PreventOverdueTicket();

```

**Đánh giá nghiệp vụ:** Áp đặt chặt chẽ các quy tắc kinh doanh (Business Rules Validations). Đảm bảo tính hợp lý và logic của dòng thời gian thực, ngăn chặn tình trạng xuất không vé hoặc lỗi nhân viên chọn nhầm suất chiếu của các ngày trước đó.

## 6.6. Viết & phân tích hiệu năng, đánh giá các câu truy vấn trong database

**1. Thêm phim mới:** Thêm bộ phim 'Lật Mặt 7' vào bảng Movie với các thông tin: Đạo diễn 'Lý Hải', Thể loại 'Gia đình', Thời lượng 120, Độ tuổi 'K', Ngày phát hành '26/04/2024'.

**Mã SQL:**

```

EXPLAIN ANALYZE
INSERT INTO Movie (title, director, genre, duration, age_rating,
releasedate)
VALUES ('Lật Mặt 7', 'Lý Hải', 'Gia đình', 120, 'K', '2024-04-26');

```

**Kết quả phân tích hiệu năng:**

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

|                                                                              |                                                                                               |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                                                              | <div> <div>QUERY PLAN</div> <div>text</div> <div>🔒</div> </div>                               |
| 1                                                                            | Insert on movie (cost=0.00..0.01 rows=0 width=0) (actual time=0.151..0.151 rows=0.00 loops=1) |
| 2                                                                            | Buffers: shared hit=6 dirtied=6                                                               |
| 3                                                                            | -> Result (cost=0.00..0.01 rows=1 width=1934) (actual time=0.036..0.036 rows=1.00 loops=1)    |
| 4                                                                            | Buffers: shared hit=1 dirtied=1                                                               |
| 5                                                                            | Planning Time: 0.057 ms                                                                       |
| 6                                                                            | Execution Time: 0.180 ms                                                                      |
| <div> <div>Total rows: 6</div> <div>Query complete 00:00:00.105</div> </div> |                                                                                               |

- **Chi phí tài nguyên (Cost: 0.00..0.01)** Chi phí thực thi gần như bằng 0. Do đây là lệnh chèn dữ liệu tĩnh (VALUES), hệ thống không phải tốn tài nguyên cho các phép quét bảng (Scan) hay kết nối (JOIN), chứng tỏ cấu trúc bảng tối ưu.
- **Thời gian thực thi (Execution Time: 0.180 ms)** Tốc độ xử lý cực kỳ nhanh. Tổng thời gian lên kế hoạch và thực thi chỉ tốn khoảng 0.2 mili-giây, mang lại tốc độ phản hồi tức thời cho người Quản lý khi thao tác trên phần mềm.
- **Hoạt động bộ nhớ (Buffers: dirtied=6)** PostgreSQL đi thẳng vào bước tạo dữ liệu và ghi nhận 6 khối bộ nhớ đệm đã được thay đổi (dirtied=6). Điều này khẳng định dữ liệu "Lật Mặt 7" đã được lưu trữ an toàn, đảm bảo tính toàn vẹn mà không gây giảm hiệu năng hệ thống.

**2. Cập nhật thông tin phim:** Sửa lại thời lượng (duration) thành 131 và nhãn độ tuổi (agerating) thành 'T18' cho bộ phim có tên là 'Lật Mặt 7'.

**Mã SQL:**

```
EXPLAIN ANALYZE
UPDATE Movie
SET duration = 131, agerating = 'T18'
WHERE title = 'Lật mặt 7';
```

## Kết quả phân tích hiệu năng:

| Data Output                     | Messages                                                                                                                     | Notifications |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------|
| <div>Showing rows: 1 to 8</div> |                                                                                                                              |               |
| <b>QUERY PLAN</b>               |                                                                                                                              |               |
| text                            |                                                                                                                              |               |
| 1                               | Update on movie (cost=0.27..8.29 rows=0 width=0) (actual time=0.071..0.071 rows=0.00 loops=1)                                |               |
| 2                               | Buffers: shared hit=2                                                                                                        |               |
| 3                               | -> Index Scan using idx_movie_search on movie (cost=0.27..8.29 rows=1 width=48) (actual time=0.067..0.067 rows=0.00 loops=1) |               |
| 4                               | Index Cond: ((title)::text = 'Lật mặt 7'::text)                                                                              |               |
| 5                               | Index Searches: 1                                                                                                            |               |
| 6                               | Buffers: shared hit=2                                                                                                        |               |
| 7                               | Planning Time: 0.164 ms                                                                                                      |               |
| 8                               | Execution Time: 0.108 ms                                                                                                     |               |

|               |                             |
|---------------|-----------------------------|
| Total rows: 8 | Query complete 00:00:00.101 |
|---------------|-----------------------------|

- **Tối ưu phương thức tìm kiếm (Index Scan)** Hệ thống kích hoạt Index Scan (qua idx\_movie\_search) để tra cứu trực tiếp vị trí dòng dữ liệu thay vì quét toàn bảng, giúp giảm tối đa chi phí truy vấn.
- **Thời gian thực thi (Execution Time: 0.108 ms)** Tốc độ xử lý siêu tốc, điều này chứng minh hệ thống được tối ưu tốt, giữ vững hiệu năng kể cả khi mở rộng quy mô dữ liệu.
- **Tiết kiệm tài nguyên bộ nhớ (Buffers: shared hit=2)** Quá trình tìm và cập nhật được truy xuất chính xác vào đúng 2 khối bộ nhớ đệm (shared hit=2), giúp tiết kiệm tài nguyên và không làm quá tải máy chủ.

**3. Lọc các phim có chi phí bản quyền cao:** Lấy danh sách tên phim, thể loại và giá mua bản quyền (importprice) của tất cả các bộ phim có giá nhập lớn hơn 100,000,000 VND, sắp xếp theo giá nhập giảm dần.

### Mã SQL:

```
EXPLAIN ANALYZE
SELECT m.title, m.genre, r.importprice
FROM Movie m
JOIN Receivinginvoice r ON m.movieid = r.movieid
WHERE r.importprice > 100000000
ORDER BY r.importprice DESC;
```

## Kết quả phân tích hiệu năng:

|                | QUERY PLAN                                                                                                         |                             |
|----------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------|
|                | text                                                                                                               | 🔒                           |
| 1              | Sort (cost=33.77..33.97 rows=81 width=52) (actual time=0.155..0.158 rows=81.00 loops=1)                            |                             |
| 2              | Sort Key: r.importprice DESC                                                                                       |                             |
| 3              | Sort Method: quicksort Memory: 30kB                                                                                |                             |
| 4              | Buffers: shared hit=20                                                                                             |                             |
| 5              | -> Hash Join (cost=3.26..31.20 rows=81 width=52) (actual time=0.046..0.129 rows=81.00 loops=1)                     |                             |
| 6              | Hash Cond: (m.movieid = r.movieid)                                                                                 |                             |
| 7              | Buffers: shared hit=20                                                                                             |                             |
| 8              | -> Seq Scan on movie m (cost=0.00..24.00 rows=500 width=50) (actual time=0.011..0.053 rows=500.00 loops=1)         |                             |
| 9              | Buffers: shared hit=19                                                                                             |                             |
| 10             | -> Hash (cost=2.25..2.25 rows=81 width=10) (actual time=0.030..0.031 rows=81.00 loops=1)                           |                             |
| 11             | Buckets: 1024 Batches: 1 Memory Usage: 12kB                                                                        |                             |
| 12             | Buffers: shared hit=1                                                                                              |                             |
| 13             | -> Seq Scan on receivinginvoice r (cost=0.00..2.25 rows=81 width=10) (actual time=0.007..0.022 rows=81.00 loops=1) |                             |
| 14             | Filter: (importprice > '100000000'::numeric)                                                                       |                             |
| 15             | Rows Removed by Filter: 19                                                                                         |                             |
| 16             | Buffers: shared hit=1                                                                                              |                             |
| 17             | Planning:                                                                                                          |                             |
| 18             | Buffers: shared hit=6                                                                                              |                             |
| 19             | Planning Time: 0.198 ms                                                                                            |                             |
| 20             | Execution Time: 0.178 ms                                                                                           |                             |
| Total rows: 20 |                                                                                                                    | Query complete 00:00:00.074 |

- **Phương thức kết nối (Hash Join & Seq Scan)** Hệ thống ưu tiên sử dụng Seq Scan kết hợp thuật toán Hash Join thay vì Index Scan. Do tỷ lệ dữ liệu cần lấy rất ít (81/100 dòng), việc tải toàn bộ bảng lên bộ nhớ để khớp mã (movieid) là chiến lược tối ưu nhất, giúp tiết kiệm chi phí đọc luân phiên từ ổ cứng.
- **Thời gian thực thi (Execution Time: 0.178 ms)** Dù phải thực hiện phép nối (JOIN) phức tạp giữa 2 bảng và có thêm điều kiện lọc, tổng thời gian thực thi vẫn cực kỳ nhanh (chưa tới 0.2 mili-giây).
- **Tối ưu sắp xếp trên RAM (Sort Method: quicksort)** Công đoạn sắp xếp hóa đơn giảm dần (ORDER BY DESC) được thực hiện hoàn toàn trên RAM bằng thuật toán quicksort. Quá trình này tiêu tốn dung lượng cực nhỏ (chỉ 30kB) và mức truy xuất đệm thấp (shared hit=20), giúp Server không bị nghẽn khi có nhiều người cùng lúc xuất báo cáo.



**4. Lọc hóa đơn nhập do một Quản lý cụ thể phụ trách:** Tìm tất cả các hóa đơn nhập hàng có tổng tiền từ 200,000,000 VND trở lên do người Quản lý có mã số managerid = 2 trực tiếp thực hiện.

**Mã SQL:**

```
EXPLAIN ANALYZE
SELECT * FROM Receivinginvoice
WHERE importprice >= 200000000
AND managerid = 2;
```

**Kết quả phân tích hiệu năng:**

| Data Output                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                             | Messages                                                          | Notifications |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|---------------|
| <div><div><div><div></div><div></div><div></div></div><div><div></div><div></div><div></div></div><div><div></div><div></div><div></div></div><div><div></div><div></div><div></div></div><div><div></div><div></div><div></div></div><div><div></div><div></div><div></div></div><div><div></div><div></div><div></div></div><div><div></div><div></div><div></div></div></div><div>SQL</div></div> |                                                                                                             | Showing rows: 1 to 8 <div><div></div><div></div><div></div></div> |               |
| QUERY PLAN                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                             |                                                                   |               |
| text                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                             |                                                                   |               |
| 1                                                                                                                                                                                                                                                                                                                                                                                                    | Seq Scan on receivinginvoice (cost=0.00..2.50 rows=4 width=30) (actual time=0.022..0.039 rows=3.00 loops=1) |                                                                   |               |
| 2                                                                                                                                                                                                                                                                                                                                                                                                    | Filter: ((importprice >= '200000000'::numeric) AND (managerid = 2))                                         |                                                                   |               |
| 3                                                                                                                                                                                                                                                                                                                                                                                                    | Rows Removed by Filter: 97                                                                                  |                                                                   |               |
| 4                                                                                                                                                                                                                                                                                                                                                                                                    | Buffers: shared hit=1                                                                                       |                                                                   |               |
| 5                                                                                                                                                                                                                                                                                                                                                                                                    | Planning:                                                                                                   |                                                                   |               |
| 6                                                                                                                                                                                                                                                                                                                                                                                                    | Buffers: shared hit=18                                                                                      |                                                                   |               |
| 7                                                                                                                                                                                                                                                                                                                                                                                                    | Planning Time: 0.207 ms                                                                                     |                                                                   |               |
| 8                                                                                                                                                                                                                                                                                                                                                                                                    | Execution Time: 0.052 ms                                                                                    |                                                                   |               |
| Total rows: 8                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                             | Query complete 00:00:00.066                                       |               |

**1. Phương thức tìm kiếm:** Hệ thống sử dụng Seq Scan (do dữ liệu bảng chỉ có 100 dòng) kết hợp bộ lọc kép (importprice và managerid). Bộ lọc hoạt động hiệu quả, loại bỏ chính xác 97 dòng không thỏa mãn với chi phí thuật toán cực thấp (cost=2.50).

**2. Thời gian thực thi (Execution Time: 0.052 ms)** Tốc độ truy xuất siêu tốc, chỉ tốn vốn vẹn **0.052 ms**. Thậm chí thời gian để hệ thống lên kế hoạch (Planning Time: 0.207 ms) còn lâu hơn cả thời gian chạy thực tế, chứng tỏ câu truy vấn quá nhẹ và tối ưu.

**3. Tối ưu bộ nhớ đệm (Buffers: shared hit=1)** Để quét toàn bộ 100 hóa đơn và trả về 3 kết quả đúng, hệ thống chỉ cần truy xuất duy nhất **1 khối bộ nhớ đệm**

(shared hit=1). Mức tiêu hao tài nguyên phần cứng gần như bằng 0, đảm bảo hiệu năng Server luôn ở mức tốt nhất.

**5. Xóa phim:** Xóa dữ liệu của bộ phim có tên 'Lật mặt 7' khỏi bảng Movie.

**Mã SQL:**

```
-- Lưu ý hệ thống: Cần đảm bảo phim này chưa có suất chiếu (ràng buộc khóa ngoại)
EXPLAIN ANALYZE
DELETE FROM Movie
WHERE title = 'Lật mặt 7';
```

**Kết quả phân tích hiệu năng:**

| Data Output                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                             | Messages | Notifications |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------|---------------|
| <div><div><div><div><div></div><div></div></div><div><div></div><div></div></div><div><div></div><div></div></div><div><div></div><div></div></div><div><div></div><div></div></div><div><div></div><div></div></div><div><div></div><div></div></div><div><div></div><div></div></div></div><div>Showing rows: 1 to 8</div><div><div></div><div></div><div></div></div><div>Pa</div></div></div> |                                                                                                                             |          |               |
| <div><div>QUERY PLAN</div><div>text</div><div></div></div>                                                                                                                                                                                                                                                                                                                                        |                                                                                                                             |          |               |
| 1                                                                                                                                                                                                                                                                                                                                                                                                 | Delete on movie (cost=0.27..8.29 rows=0 width=0) (actual time=0.054..0.054 rows=0.00 loops=1)                               |          |               |
| 2                                                                                                                                                                                                                                                                                                                                                                                                 | Buffers: shared hit=2                                                                                                       |          |               |
| 3                                                                                                                                                                                                                                                                                                                                                                                                 | -> Index Scan using idx_movie_search on movie (cost=0.27..8.29 rows=1 width=6) (actual time=0.050..0.050 rows=0.00 loops=1) |          |               |
| 4                                                                                                                                                                                                                                                                                                                                                                                                 | Index Cond: ((title)::text = 'Lật mặt 7'::text)                                                                             |          |               |
| 5                                                                                                                                                                                                                                                                                                                                                                                                 | Index Searches: 1                                                                                                           |          |               |
| 6                                                                                                                                                                                                                                                                                                                                                                                                 | Buffers: shared hit=2                                                                                                       |          |               |
| 7                                                                                                                                                                                                                                                                                                                                                                                                 | Planning Time: 0.140 ms                                                                                                     |          |               |
| 8                                                                                                                                                                                                                                                                                                                                                                                                 | Execution Time: 0.088 ms                                                                                                    |          |               |
| <div><div>Total rows: 8</div><div>Query complete 00:00:00.095</div></div>                                                                                                                                                                                                                                                                                                                         |                                                                                                                             |          |               |

- **Tối ưu định vị dữ liệu (Index Scan)** Hệ thống sử dụng trực tiếp chỉ mục idx\_movie\_search để tra cứu vị trí bộ phim cần xóa. Việc bỏ qua phương pháp quét toàn bảng giúp chi phí thuật toán được kiểm soát ở mức cực kỳ thấp (cost=0.27..8.29).
- **Thời gian thực thi (Execution Time: 0.088 ms)** Quá trình xóa diễn ra chớp nhoáng chỉ trong **0.088 ms**. Tốc độ xử lý siêu tốc này rất quan trọng để tránh tình trạng "khóa bảng" (Table Lock) quá lâu, không làm ảnh hưởng đến các giao dịch mua vé đang diễn ra.

- **Tiết kiệm tài nguyên (Buffers: shared hit=2)** CSDL chỉ cần truy xuất chính xác 2 khối bộ nhớ đệm (shared hit=2) để hoàn tất thao tác xác định và xóa dòng dữ liệu. Quá trình xử lý vô cùng gọn nhẹ, không gây áp lực lên dung lượng RAM của máy chủ.

**6. Danh sách khách hàng VIP:** Liệt kê họ tên khách hàng, tên đăng nhập (username) và số điểm tích lũy của 10 khách hàng có điểm tích lũy cao nhất hệ thống.

**Mã SQL:**

```
-- Sử dụng LIMIT để lấy Top 10 người có điểm số (rewardpoints) cao nhất
EXPLAIN ANALYZE
SELECT c.fullname, ma.username, ma.rewardpoints
FROM Customer c
JOIN Movie_Account ma ON c.customerid = ma.customerid
ORDER BY ma.rewardpoints DESC
LIMIT 10;
```

**Kết quả phân tích hiệu năng:**

|                | QUERY PLAN                                                                                                                  |                                                                                       |
|----------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|                | text                                                                                                                        |  |
| 1              | Limit (cost=0.55..5.35 rows=10 width=37) (actual time=0.061..0.091 rows=10.00 loops=1)                                      |                                                                                       |
| 2              | Buffers: shared hit=39 read=2                                                                                               |                                                                                       |
| 3              | -> Nested Loop (cost=0.55..240.69 rows=500 width=37) (actual time=0.060..0.089 rows=10.00 loops=1)                          |                                                                                       |
| 4              | Buffers: shared hit=39 read=2                                                                                               |                                                                                       |
| 5              | -> Index Scan using idx_account_points on movie_account ma (cost=0.27..63.77 rows=500 width=24) (actual time=0.052..0.0...  |                                                                                       |
| 6              | Index Searches: 1                                                                                                           |                                                                                       |
| 7              | Buffers: shared hit=9 read=2                                                                                                |                                                                                       |
| 8              | -> Index Scan using customer_pkey on customer c (cost=0.27..0.35 rows=1 width=21) (actual time=0.002..0.002 rows=1.00 lo... |                                                                                       |
| 9              | Index Cond: (customerid = ma.customerid)                                                                                    |                                                                                       |
| 10             | Index Searches: 10                                                                                                          |                                                                                       |
| 11             | Buffers: shared hit=30                                                                                                      |                                                                                       |
| 12             | Planning:                                                                                                                   |                                                                                       |
| 13             | Buffers: shared hit=26 read=1                                                                                               |                                                                                       |
| 14             | Planning Time: 1.348 ms                                                                                                     |                                                                                       |
| 15             | Execution Time: 0.121 ms                                                                                                    |                                                                                       |
| Total rows: 15 |                                                                                                                             | Query complete 00:00:00.059                                                           |

- **Tối ưu sắp xếp và kết nối kép:** Hệ thống đã tận dụng xuất sắc idx\_account\_points vừa tạo để quét điểm số trực tiếp từ cao xuống thấp. Sau đó, nó kết hợp với Khóa chính (customer\_pkey) thông qua thuật toán Nested Loop để nối bảng lấy tên khách hàng mà không tốn một chút chi phí sắp xếp thủ công (Sort) nào.
- **Thời gian thực thi (Execution Time: 0.121 ms)** Tốc độ xử lý siêu tốc, chỉ mất **0.121 mili-giây**. Việc đánh Index đúng mục tiêu giúp thuật toán đạt ngưỡng phản hồi tức thời (real-time), sẵn sàng đáp ứng mở rộng lên hàng triệu tài khoản người dùng.
- **Hiệu quả tuyệt đối của Limit (Buffers: shared hit=39)** Sự kết hợp hoàn hảo giữa Index Scan và Limit 10 giúp hệ thống hiểu rằng: Chỉ cần lấy đủ 10 người đầu tiên trên cây Index là dừng ngay lập tức. Quá trình này chỉ truy xuất đúng 39 khối bộ nhớ đệm (shared hit=39), chặn đứng hoàn toàn việc lãng phí tài nguyên RAM.

**7. Thống kê lượng vé bán theo phim:** Kết quả hiển thị tên phim (title) và "Tổng số vé đã bán", sắp xếp theo số lượng giảm dần.

**Mã SQL:**

```
-- Kết nối 3 bảng Movie, MovieShow và Ticket để đếm số vé bán theo phim
EXPLAIN ANALYZE
SELECT m.title, COUNT(t.ticketid) AS "Tổng số vé đã bán"
FROM Movie m
JOIN MovieShow ms ON m.movieid = ms.movieid
JOIN Ticket t ON ms.movieshowid = t.movieshowid
GROUP BY m.title
ORDER BY COUNT(t.ticketid) DESC;
```

**Kết quả phân tích hiệu năng:**

| Data Output           | Messages                                                                                                       | Notifications |
|-----------------------|----------------------------------------------------------------------------------------------------------------|---------------|
| Showing rows: 1 to 30 |                                                                                                                | Page          |
| QUERY PLAN            |                                                                                                                |               |
| text                  |                                                                                                                |               |
| 18                    | Buckets: 1024 Batches: 1 Memory Usage: 20kB                                                                    |               |
| 19                    | Buffers: shared hit=4                                                                                          |               |
| 20                    | -> Seq Scan on moviewshow ms (cost=0.00..9.00 rows=500 width=8) (actual time=0.007..0.072 rows=500.00 loops=1) |               |
| 21                    | Buffers: shared hit=4                                                                                          |               |
| 22                    | -> Hash (cost=24.00..24.00 rows=500 width=38) (actual time=0.175..0.175 rows=500.00 loops=1)                   |               |
| 23                    | Buckets: 1024 Batches: 1 Memory Usage: 44kB                                                                    |               |
| 24                    | Buffers: shared hit=19                                                                                         |               |
| 25                    | -> Seq Scan on movie m (cost=0.00..24.00 rows=500 width=38) (actual time=0.016..0.109 rows=500.00 loops=1)     |               |
| 26                    | Buffers: shared hit=19                                                                                         |               |
| 27                    | Planning:                                                                                                      |               |
| 28                    | Buffers: shared hit=24                                                                                         |               |
| 29                    | Planning Time: 0.422 ms                                                                                        |               |
| 30                    | Execution Time: 0.816 ms                                                                                       |               |

- **Chiến lược quét toàn bảng có chủ đích (Seq Scan)** Dù đã đánh Index khóa ngoại, hệ thống vẫn ưu tiên dùng Seq Scan cho bảng Movie và MovieShow. Đây là quyết định tối ưu hoàn toàn chính xác của hệ thống, bởi yêu cầu bài toán là thống kê toàn bộ phim (không có mệnh đề WHERE lọc dữ liệu). Khi cần truy xuất 100% dữ liệu bảng, quét tuần tự sẽ nhanh hơn việc tra cứu qua cây chỉ mục.
- **Tối ưu kết nối bằng bảng băm trên RAM (Hash Join)** Thay vì dùng Index để nối từng dòng (Nested Loop) cực kỳ chậm cho dữ liệu lớn, PostgreSQL đã đẩy toàn bộ 500 dòng phim lên bộ nhớ RAM để tạo một Bảng băm (Hash). Quá trình này chỉ tốn 44kB Memory Usage, giúp phép nối (JOIN) 3 bảng diễn ra mượt mà không gây nghẽn cổ chai ở ổ cứng.
- **Thời gian thực thi cực thấp (Execution Time: 0.816 ms)** Phải thực hiện kết nối 3 bảng phức tạp và đếm tổng (COUNT) gom nhóm (GROUP BY), nhưng tổng thời gian thực thi chỉ mất chưa tới **1 mili-giây (0.816 ms)**

**8. Hiệu suất bán bắp/nước của nhân viên:** Đếm tổng số lượng sản phẩm F&B mà từng nhân viên thu ngân đã bán được. Hiện thị họ tên nhân viên và "Tổng sản phẩm F&B đã bán".

**Mã SQL:**

```
-- Sử dụng hàm SUM() để cộng dồn số lượng sản phẩm F&B trong chi tiết hóa đơn
EXPLAIN ANALYZE
SELECT s.fullname, SUM(bd.quantity) AS "Tổng sản phẩm F&B đã bán"
FROM Staff s
JOIN Bill b ON s.staffid = b.staffid
JOIN Bill_Detail bd ON b.billid = bd.billid
GROUP BY s.fullname;
```

## Kết quả phân tích hiệu năng:

|                | QUERY PLAN                                                                                                       |                             |
|----------------|------------------------------------------------------------------------------------------------------------------|-----------------------------|
|                | text                                                                                                             |                             |
| 6              | Hash Cond: (b.staffid = s.staffid)                                                                               |                             |
| 7              | Buffers: shared hit=13                                                                                           |                             |
| 8              | -> Hash Join (cost=16.25..30.78 rows=753 width=8) (actual time=0.147..0.316 rows=753.00 loops=1)                 |                             |
| 9              | Hash Cond: (bd.billid = b.billid)                                                                                |                             |
| 10             | Buffers: shared hit=10                                                                                           |                             |
| 11             | -> Seq Scan on bill_detail bd (cost=0.00..12.53 rows=753 width=8) (actual time=0.009..0.047 rows=753.00 loops=1) |                             |
| 12             | Buffers: shared hit=5                                                                                            |                             |
| 13             | -> Hash (cost=10.00..10.00 rows=500 width=8) (actual time=0.133..0.133 rows=500.00 loops=1)                      |                             |
| 14             | Buckets: 1024 Batches: 1 Memory Usage: 28kB                                                                      |                             |
| 15             | Buffers: shared hit=5                                                                                            |                             |
| 16             | -> Seq Scan on bill b (cost=0.00..10.00 rows=500 width=8) (actual time=0.009..0.069 rows=500.00 loops=1)         |                             |
| 17             | Buffers: shared hit=5                                                                                            |                             |
| 18             | -> Hash (cost=4.50..4.50 rows=150 width=21) (actual time=0.066..0.066 rows=150.00 loops=1)                       |                             |
| 19             | Buckets: 1024 Batches: 1 Memory Usage: 17kB                                                                      |                             |
| 20             | Buffers: shared hit=3                                                                                            |                             |
| 21             | -> Seq Scan on staff s (cost=0.00..4.50 rows=150 width=21) (actual time=0.021..0.040 rows=150.00 loops=1)        |                             |
| 22             | Buffers: shared hit=3                                                                                            |                             |
| 23             | Planning:                                                                                                        |                             |
| 24             | Buffers: shared hit=53 read=6                                                                                    |                             |
| 25             | Planning Time: 2.695 ms                                                                                          |                             |
| 26             | Execution Time: 0.750 ms                                                                                         |                             |
| Total rows: 26 |                                                                                                                  | Query complete 00:00:00.102 |

- **Quét toàn cục và kết nối đa tầng (Seq Scan & Hash Join)** Do bài toán yêu cầu tính tổng cho *tất cả* nhân viên (không có mệnh đề WHERE), hệ thống đã chủ động dùng Seq Scan và kết nối 3 bảng bằng thuật toán Hash Join. Đây là quyết định tối ưu tuyệt đối của bộ máy (Query Planner), vì bốc toàn bộ dữ liệu lên để khớp một lượt sẽ nhanh hơn rất nhiều so với việc tra cứu Index từng dòng riêng lẻ.

- **Tối ưu xử lý trực tiếp trên RAM (Memory Usage)** Thay vì đọc/ghi luân phiên trên ổ cứng làm chậm hệ thống, PostgreSQL đẩy dữ liệu lên RAM để tạo các bảng băm (Hash). Quá trình này tiêu tốn dung lượng cực kỳ khiêm tốn (chỉ **17kB** và **28kB** Memory Usage), giúp phép nối hàng trăm hóa đơn diễn ra mượt mà, không gây áp lực lên phần cứng Server.
- **Thời gian thực thi chớp nhoáng (Execution Time: 0.750 ms)** Mặc dù phải thực hiện logic gom nhóm (GROUP BY) và cộng dồn (SUM) qua 3 bảng khác nhau, tổng thời gian chạy thực tế chỉ tốn chưa tới **1 mili-giây (0.750 ms)**. Hiệu năng xuất sắc này đảm bảo tính năng báo cáo doanh thu luôn hoạt động mượt mà, đáp ứng chuẩn Real-time cho người quản lý.

**9. Nhân viên không phát sinh giao dịch:** Sử dụng Subquery để tìm thông tin các nhân viên (Staff) chưa từng lập bất kỳ hóa đơn (Bill) nào trong tháng hiện tại.

**Mã SQL:**

```
-- Dùng NOT IN đối chiếu với tập hợp các staffid có lập hóa đơn trong
tháng/năm nay
EXPLAIN ANALYZE
SELECT staffid, fullname
FROM Staff
WHERE staffid NOT IN (
    SELECT DISTINCT staffid
    FROM Bill
    WHERE EXTRACT(MONTH FROM issuedate) = EXTRACT(MONTH FROM
CURRENT_DATE)
    AND EXTRACT(YEAR FROM issuedate) = EXTRACT(YEAR FROM
CURRENT_DATE)
);
```

**Kết quả phân tích hiệu năng:**

Data Output

Messages

Notifications

<

- Khai thác sức mạnh của Function-Based Index:** Hệ thống kích hoạt Index Scan using idx\_bill\_month\_year, đi thẳng vào cây chỉ mục để lấy các hóa đơn đúng tháng/năm hiện tại với chi phí truy xuất cực thấp (cost bắt đầu từ 0.28).
- Tối ưu xử lý Subquery (Hashed SubPlan & Unique)** Để giải quyết lệnh NOT IN, CSDL đã dùng thuật toán quicksort siêu nhẹ (chỉ tốn **25kB** RAM) để sắp xếp và loại bỏ trùng lặp (Unique). Tập danh sách nhân viên này được băm (Hashed) để đối chiếu ngược, giúp loại bỏ chính xác 146 người đã lập hóa đơn một cách nhanh gọn.
- Thời gian thực thi:** Tổng thời gian giải quyết một bài toán truy vấn lồng (Subquery) phức tạp chỉ mất vỏn vẹn **0.315 mili-giây**. Tốc độ này chứng minh việc kết hợp đúng loại Index đã giúp thuật toán hoạt động hoàn hảo, loại bỏ hoàn toàn rủi ro nghẽn cổ chai cho máy chủ.



**10. Suất chiếu "cháy vé" (Đông khách nhất):** Dùng Subquery để tra cứu thông tin suất chiếu (Mã suất chiếu, Tên phim, Ngày chiếu) sở hữu số lượng vé bán ra đạt mức cao nhất so với tất cả các suất chiếu khác.

**Mã SQL:**

```
EXPLAIN ANALYZE
SELECT ms.movieshowid, m.title, COUNT(t.ticketid) AS "Số vé bán ra"
FROM MovieShow ms
JOIN Movie m ON ms.movieid = m.movieid
JOIN Ticket t ON ms.movieshowid = t.movieshowid
GROUP BY ms.movieshowid, m.title
HAVING COUNT(t.ticketid) = (
    -- Subquery lấy ra số lượng vé cao nhất của một suất chiếu bất kỳ
    SELECT MAX(ticket_count)
    FROM (
        SELECT COUNT(ticketid) AS ticket_count
        FROM Ticket
        GROUP BY movieshowid
    ) AS Sub
);
```

**Kết quả phân tích hiệu năng:**

|                | QUERY PLAN                                                                                                    |                                                                                       |
|----------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|                | text                                                                                                          |  |
| 26             | Buffers: shared hit=4                                                                                         |                                                                                       |
| 27             | -> Seq Scan on movieshow ms (cost=0.00..9.00 rows=500 width=8) (actual time=0.006..0.046 rows=500.00 loops=1) |                                                                                       |
| 28             | Buffers: shared hit=4                                                                                         |                                                                                       |
| 29             | -> Hash (cost=24.00..24.00 rows=500 width=38) (actual time=0.171..0.171 rows=500.00 loops=1)                  |                                                                                       |
| 30             | Buckets: 1024 Batches: 1 Memory Usage: 44kB                                                                   |                                                                                       |
| 31             | Buffers: shared hit=19                                                                                        |                                                                                       |
| 32             | -> Seq Scan on movie m (cost=0.00..24.00 rows=500 width=38) (actual time=0.008..0.096 rows=500.00 loops=1)    |                                                                                       |
| 33             | Buffers: shared hit=19                                                                                        |                                                                                       |
| 34             | Planning:                                                                                                     |                                                                                       |
| 35             | Buffers: shared hit=24                                                                                        |                                                                                       |
| 36             | Planning Time: 0.510 ms                                                                                       |                                                                                       |
| 37             | Execution Time: 1.250 ms                                                                                      |                                                                                       |
| Total rows: 37 |                                                                                                               | Query complete 00:00:00.104                                                           |

- **Quét toàn cục và băm dữ liệu (Seq Scan & Hash)** Hệ thống sử dụng Seq Scan quét toàn bộ 500 dòng của bảng Movie và MovieShow để phục vụ cho thuật toán Hash Join.

- **Tiết kiệm tối đa dung lượng RAM (Memory Usage: 44kB)** Quá trình tạo bảng băm (Hash) cho bảng phim diễn ra cực kỳ nhẹ bèn, chỉ tiêu tốn vón vẹn **44kB** dung lượng RAM.
- **Thời gian thực thi (Execution Time: 1.250 ms):** Con số này minh chứng rõ ràng cho việc cấu trúc cơ sở dữ liệu đã được thiết kế để sẵn sàng xuất các báo cáo thi đua/thống kê phức tạp theo thời gian thực (real-time).

## VII. Hướng dẫn cài đặt & sử dụng hệ thống quản lý

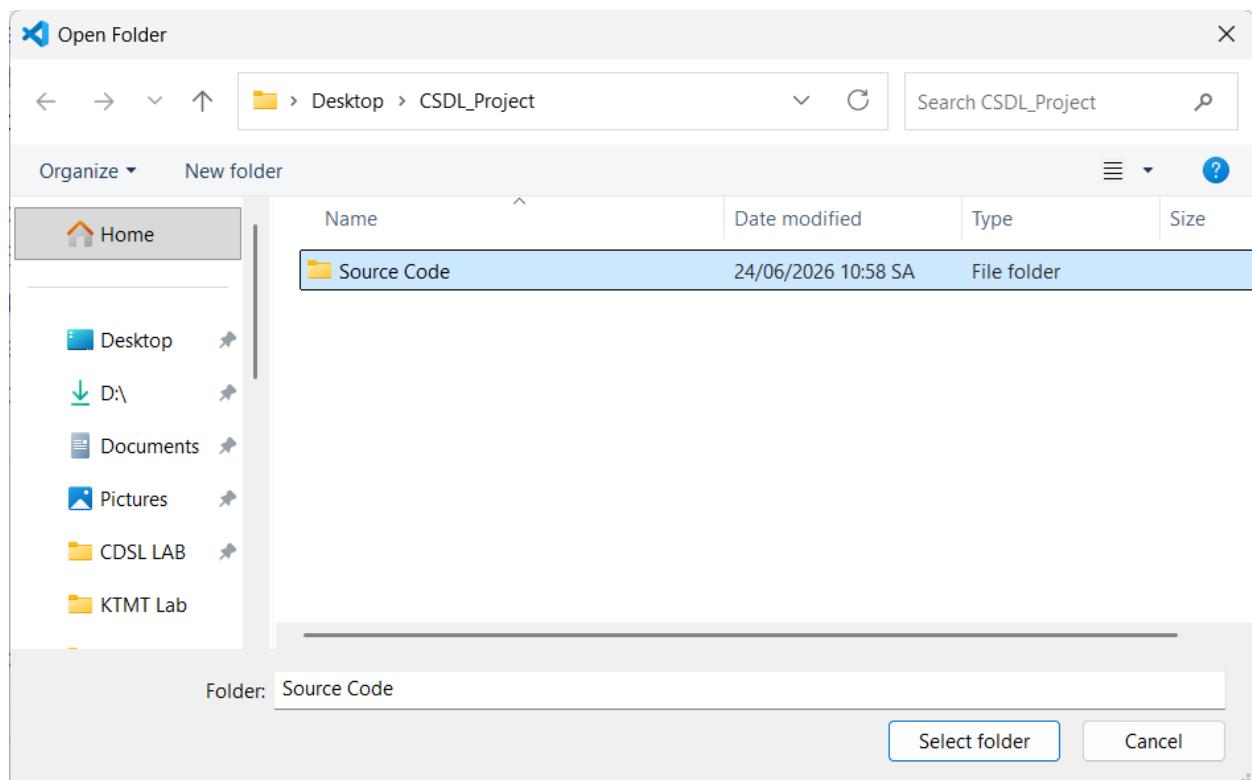
### 7.1. Yêu cầu cài đặt trước khi khởi chạy

- Java Development Kit (JDK): Phiên bản 21 (hoặc tương thích từ Java 11 trở lên).
- Cơ sở dữ liệu: PostgreSQL và công cụ quản trị pgAdmin4.
- Thiết lập thư viện PostgreSQL postgresql-42.x.x.jar trong cấu hình môi trường của máy

### 7.2. Cài đặt & cấu hình

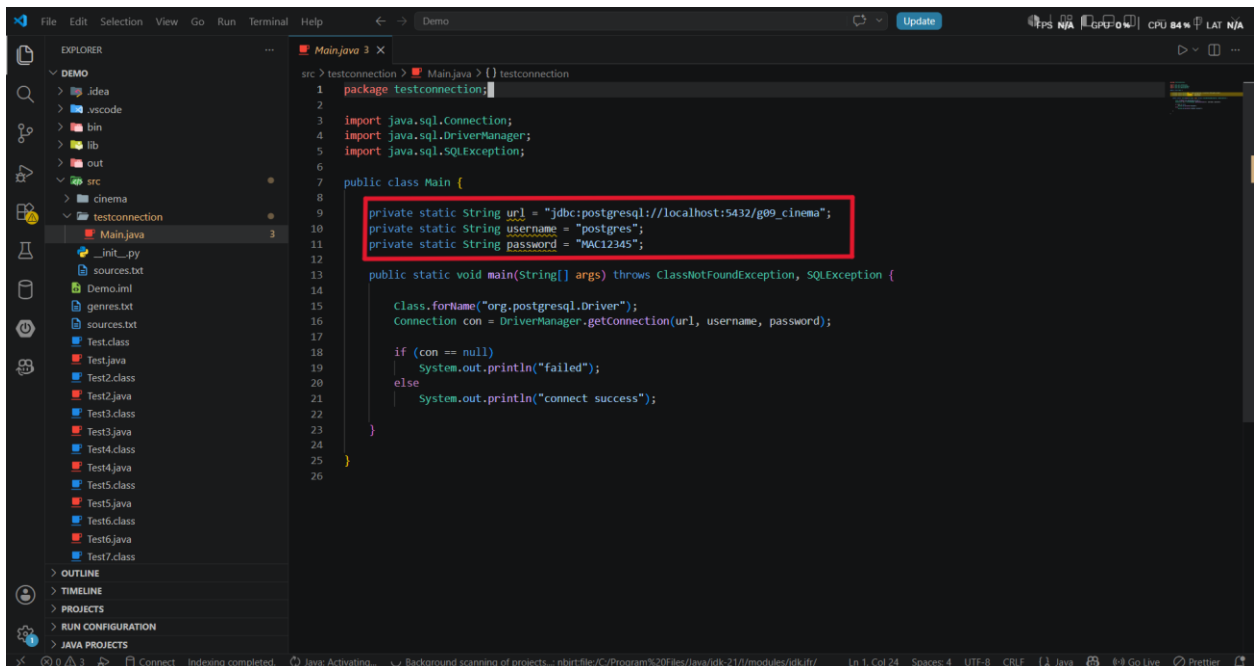
#### Bước 1: Mở dự án bằng IDE (VS Code, IntelliJ, ..)

1. Giải nén thư mục chứa mã nguồn dự án (ví dụ: Source Code).
2. Chọn File > Open Folder..., sau đó trở đến thư mục dự án vừa giải nén.



## Bước 2: Cập nhật thông tin kết nối đảm bảo Java có thể giao tiếp với cơ sở dữ liệu trên máy

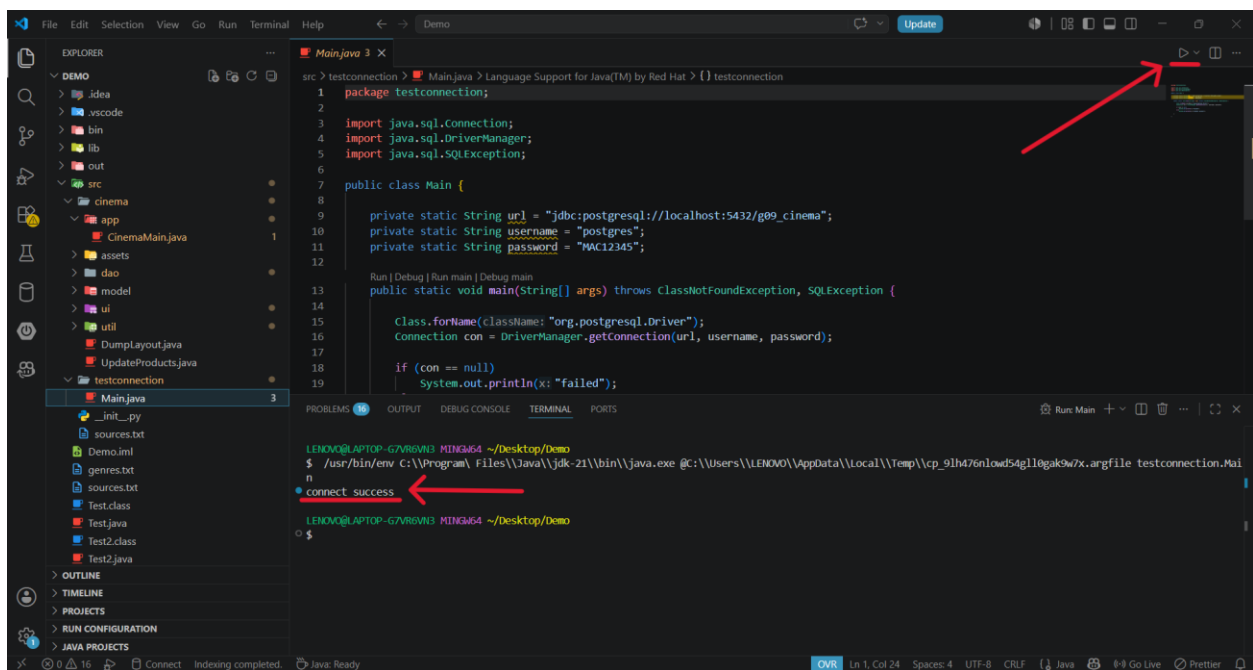
1. Mở file [src/testconnection/Main.java].
2. Kiểm tra và chỉnh sửa lại các thông số kết nối cho khớp với tài khoản pgAdmin4. Cụ thể:
  - **Đường dẫn (URL):** jdbc:postgresql://localhost:5432/g09\_cinema
  - **Tên đăng nhập (Username):** postgres
  - **Mật khẩu (Password):** admin (hãy đổi thành mật khẩu PostgreSQL thực tế trên máy).



```
1 package testconnection;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class Main {
8
9     private static String url = "jdbc:postgresql://localhost:5432/g09_cinema";
10    private static String username = "postgres";
11    private static String password = "MAC12345";
12
13    public static void main(String[] args) throws ClassNotFoundException, SQLException {
14
15        Class.forName("org.postgresql.Driver");
16        Connection con = DriverManager.getConnection(url, username, password);
17
18        if (con == null)
19            System.out.println("failed");
20        else
21            System.out.println("connect success");
22    }
23
24 }
25
26 }
```

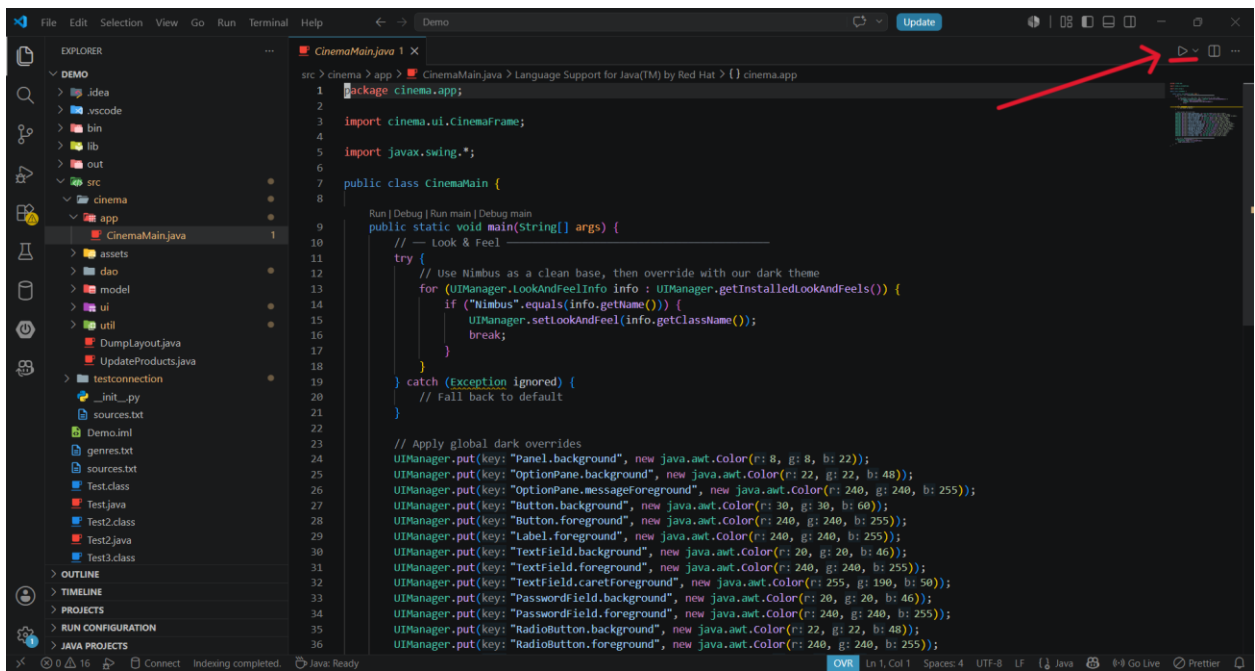
### Bước 3: Kiểm tra kết nối Cơ sở dữ liệu

1. Tìm đến file src/testconnection/Main.java.
2. Bấm Run để chạy.
3. Quan sát cửa sổ Terminal ở phía dưới màn hình. Nếu in ra dòng chữ **"connect success"**, hệ thống đã cấu hình thành công.

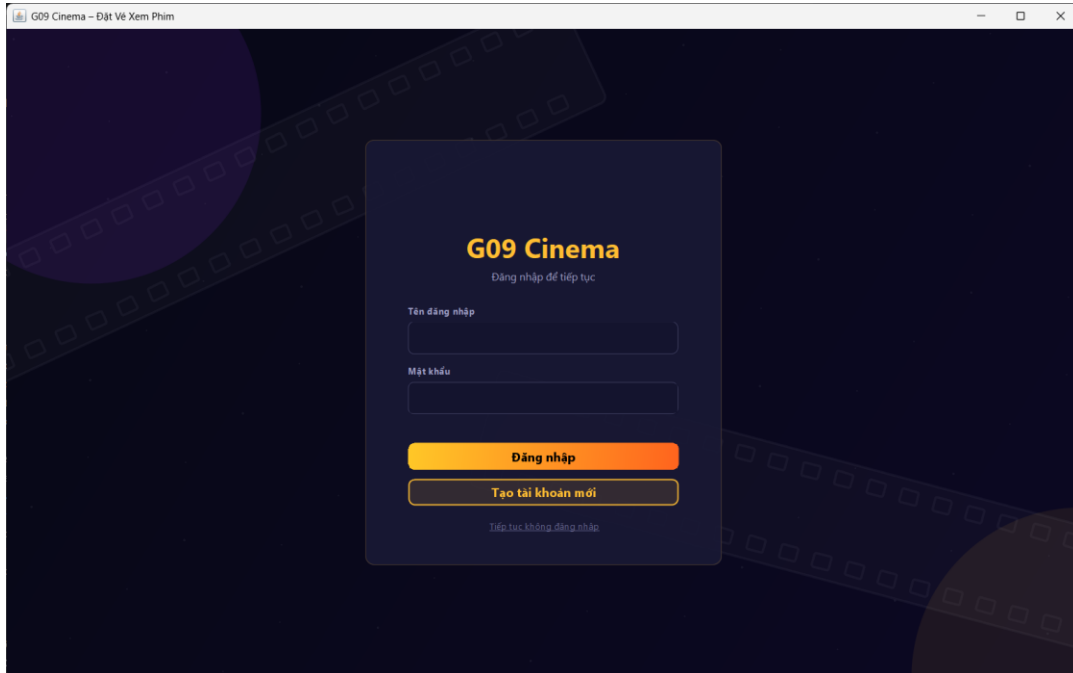


## Bước 4: Khởi chạy ứng dụng chính

1. Tìm đến [src/cinema/app/CinemaMain.java].
2. Bấm Run để chạy.
3. Giao diện đồ họa của phần mềm quản lý rạp chiếu phim sẽ xuất hiện và có thể bắt đầu sử dụng.

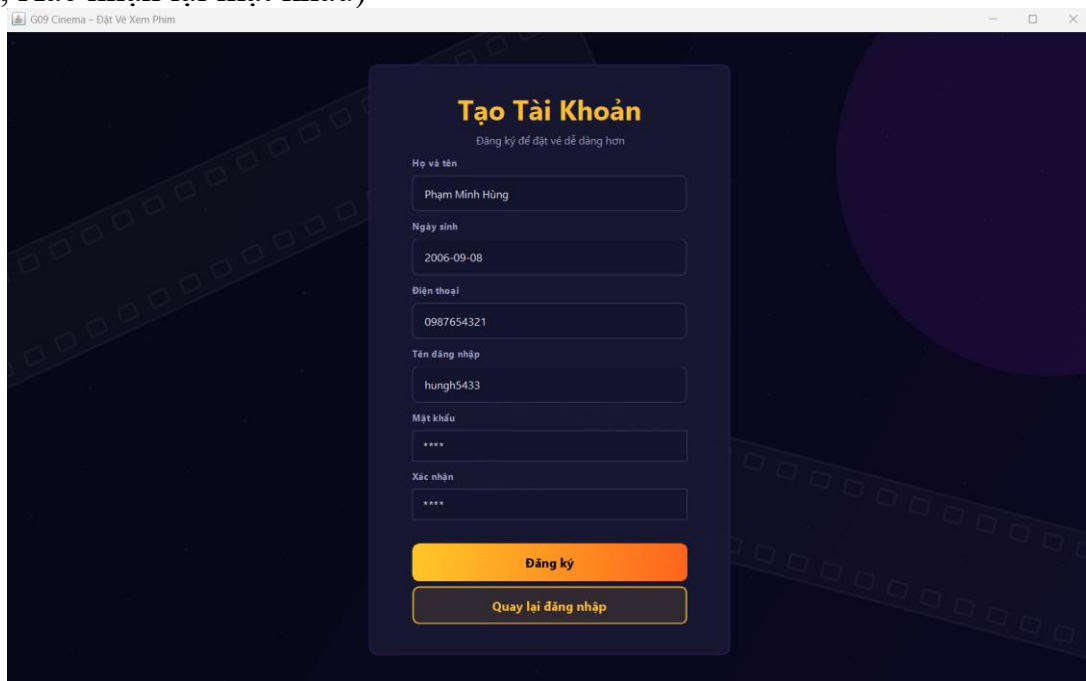


## 7.3. Hướng dẫn sử dụng

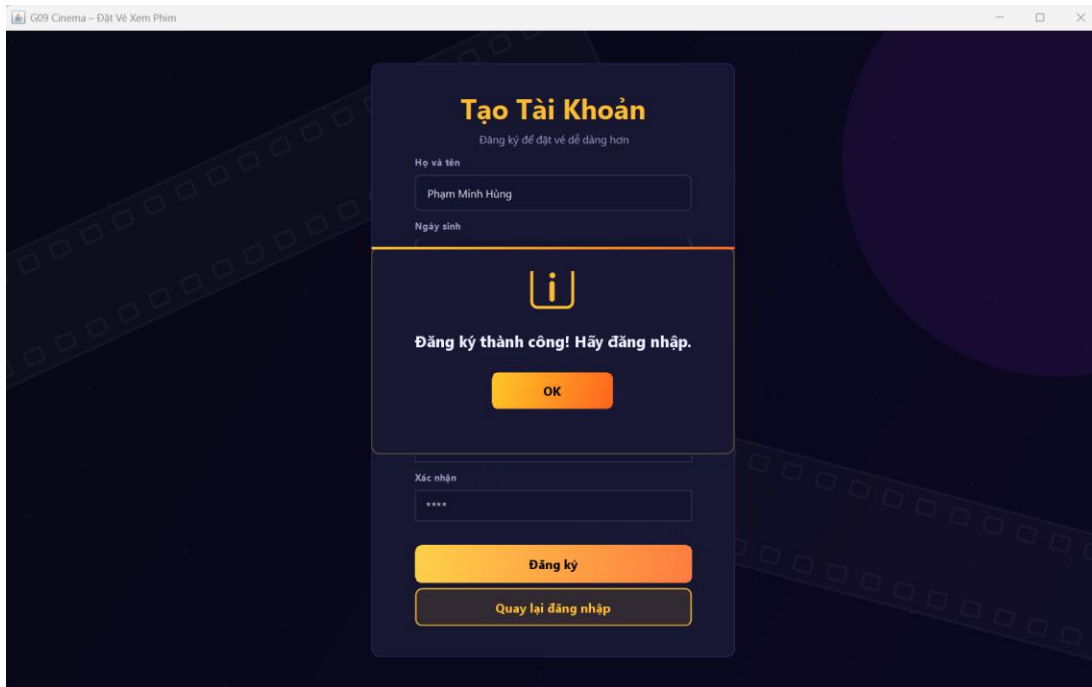


*(Giao diện khởi động của app / Đăng nhập)*

### 1. Nhập thông tin vào các trường bắt buộc (Họ & Tên, Tên đăng nhập, Mật khẩu, Xác nhận lại mật khẩu)

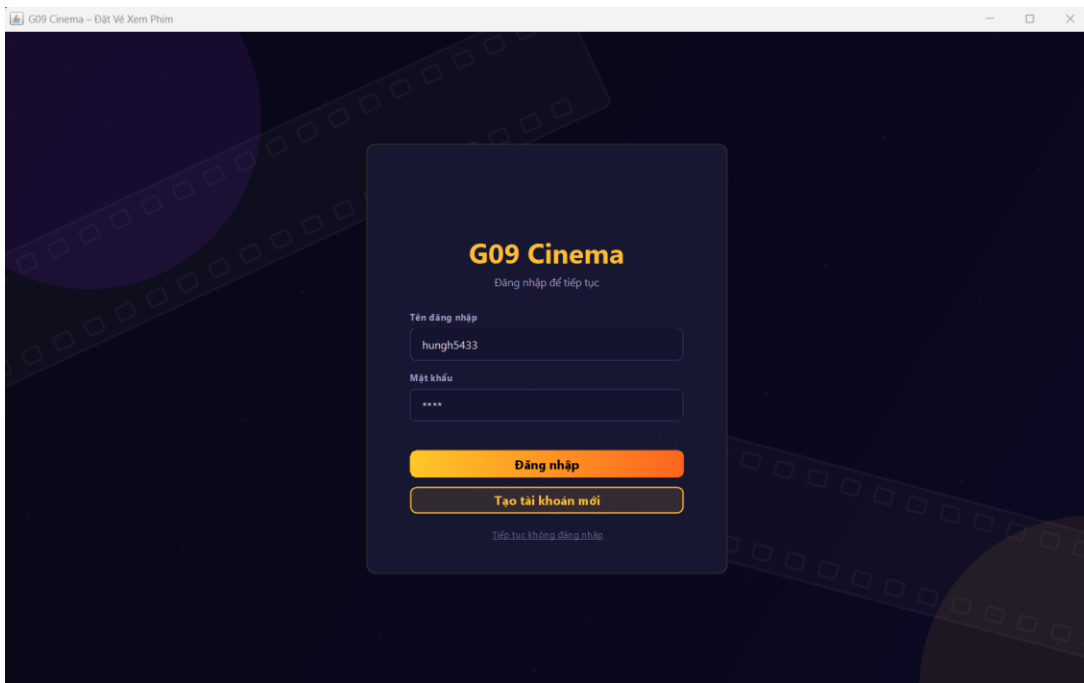


*(Giao diện đăng ký)*



*(thông báo đã đăng kí thành công)*

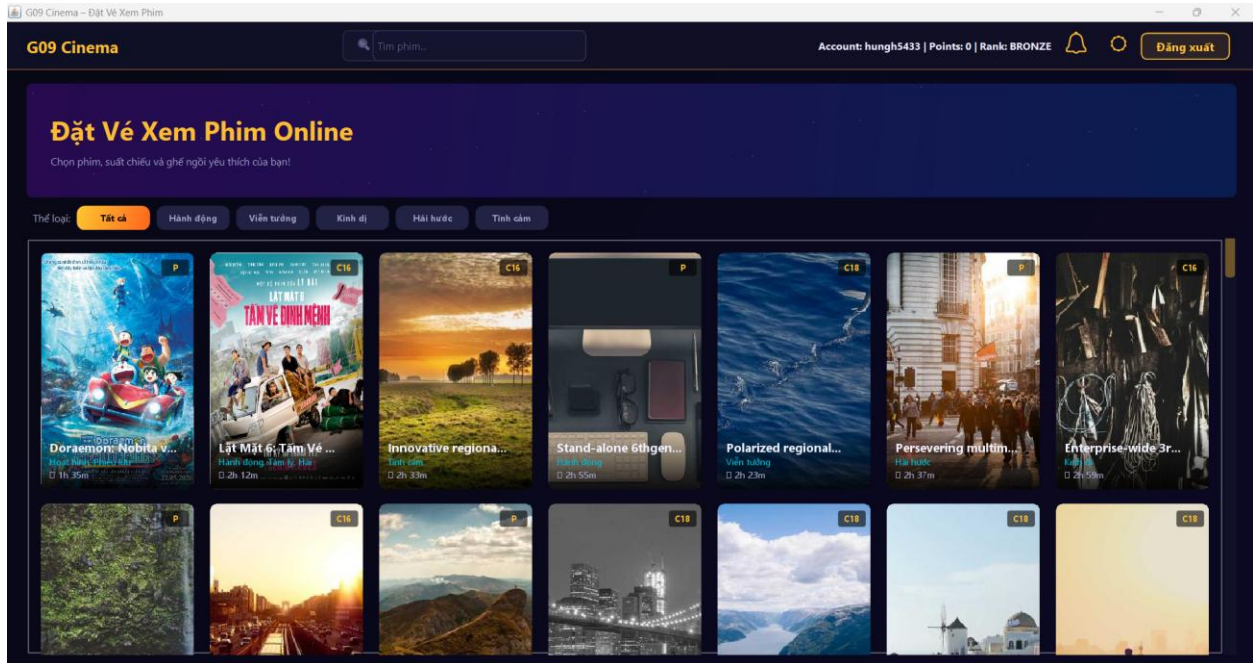
## 2. Sau khi đăng ký, cần nhớ Tên đăng nhập và Mật khẩu để đăng nhập



*(Đăng nhập)*



3. Trang chủ sau khi đăng nhập bao gồm các bộ phim đang được chiếu, phân theo từng thể loại, thanh công cụ tra cứu phim, thông tin user, thông báo cập nhật phim hay nút chuyển đổi sáng/tối.



(Ảnh minh họa)

Ngoài ra cũng có thể tiếp tục mà không đăng nhập (với **tư cách khách**), nhưng chỉ có thể xem những khung giờ và phim đang chiếu -> cần phải đăng nhập để có thể đặt vé.

## 4. Hướng dẫn đặt vé

Click vào phim muốn đặt để xem các thông tin của phim

The screenshot shows the movie details page for 'Doraemon: Nobita và lâu đài dưới đáy biển' on the G99 Cinema website. The page includes a large image of the movie's underwater scene, a synopsis in Vietnamese, and showtimes for two dates: 19/06 at 14:00 (Phòng 02) and 20/06 at 09:30 (Phòng 03). The synopsis describes Nobita and his friends' adventure in the underwater world of Atlantis, where they encounter Poseidon and his army.

Chúng ta nhất định sẽ hiểu nhau.  
Nơi đáy biển và tận đáy tâm hồn.

### Doraemon: Nobita và lâu đài dưới đáy biển

Thể loại: Hoạt hình, Phiêu lưu | Thời lượng: 1h 35m | Ngôn ngữ: Tiếng Nhật | Khởi chiếu: 19/06/2026

Đạo diễn: Tsutomu Shibayama  
Diễn viên: Kaneta Kimotsuki, Kazuya Tatekabe, Michiko Nomura, Nobuyo Oyama, Noriko Ohara

#### Nội dung phim

Kỳ nghỉ hè đến, Nobita và nhóm bạn quyết định nhờ Doraemon sử dụng bảo bối để đi cắm trại và thám hiểm dưới đáy đại dương mê mông. Tại đây, họ tình cờ chạm trán với cư dân của vương quốc dưới đáy biển Mu và biết được một bí mật khủng khiếp. Poseidon - thủ lĩnh của quốc gia Atlantis tà ác đang lên kế hoạch kích hoạt hệ thống vũ khí hủy diệt hàng loạt nhằm san phẳng toàn bộ thế giới trên cạn. Nhóm bạn Doraemon cùng nhau bước vào một cuộc chiến sinh tử dưới lòng đại dương sâu thẳm để giải cứu nhân loại khỏi thảm họa diệt vong.

#### Chọn suất chiếu

| Ngày  | Thời gian | Phòng    |
|-------|-----------|----------|
| 19/06 | 14:00     | Phòng 02 |
| 20/06 | 09:30     | Phòng 03 |

(Thông tin phim)

Sau khi chọn suất chiếu, giao diện đặt ghế sẽ hiện ra

The screenshot shows the seat selection interface for the movie 'Doraemon: Nobita và lâu đài dưới đáy biển'. The interface displays a grid of seats arranged in rows A through F and columns 1 through 10. Seats are color-coded: green for standard, purple for VIP, and red for Sweet Box. A legend on the right side explains the color coding and prices. The total price is currently 0 VND, and there is a 'Tiếp tục' (Continue) button.

### Doraemon: Nobita và lâu đài dưới đáy biển

2026-06-19 | 14:00 - 15:35 | Phòng 02

— MÀN HÌNH —

A 1 2 3 4 5 6 7 8 9 10

B 1 2 3 4 5 6 7 8 9 10

C 1 2 3 4 5 6 7 8 9 10

D 1 2 3 4 5 6 7 8 9 10

E 1 2 3 4 5 6 7 8 9 10

F 1 2 3 4 5 6 7 8 9 10

#### Chú thích

- Ghế trống (Standard) 85.000đ
- VIP 130.000đ
- Sweet Box 200.000đ
- Đang chọn
- Đã đặt

#### Ghế đã chọn

Chưa chọn ghế

**Tổng: 0 đ**

**Tiếp tục**

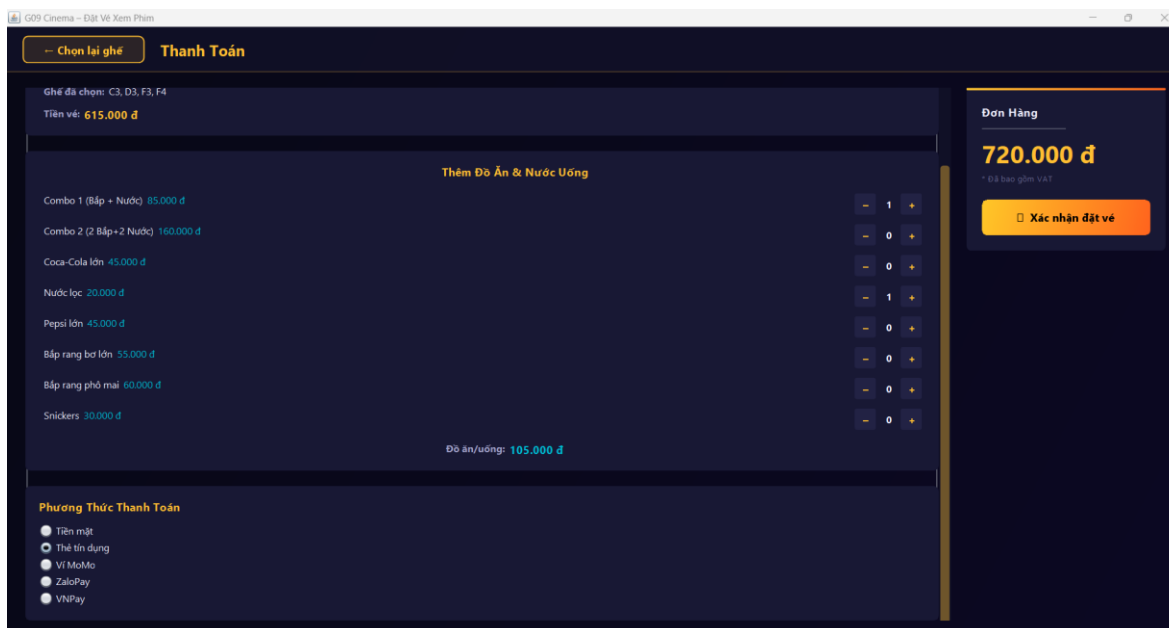
(Giao diện đặt ghế)

Xem các trạng thái **ghế ngồi ở bên phải**, khi chọn các ghế ngồi, **tổng giá ghế sẽ được hiển thị**, bấm tiếp tục để đặt các dịch vụ đi kèm



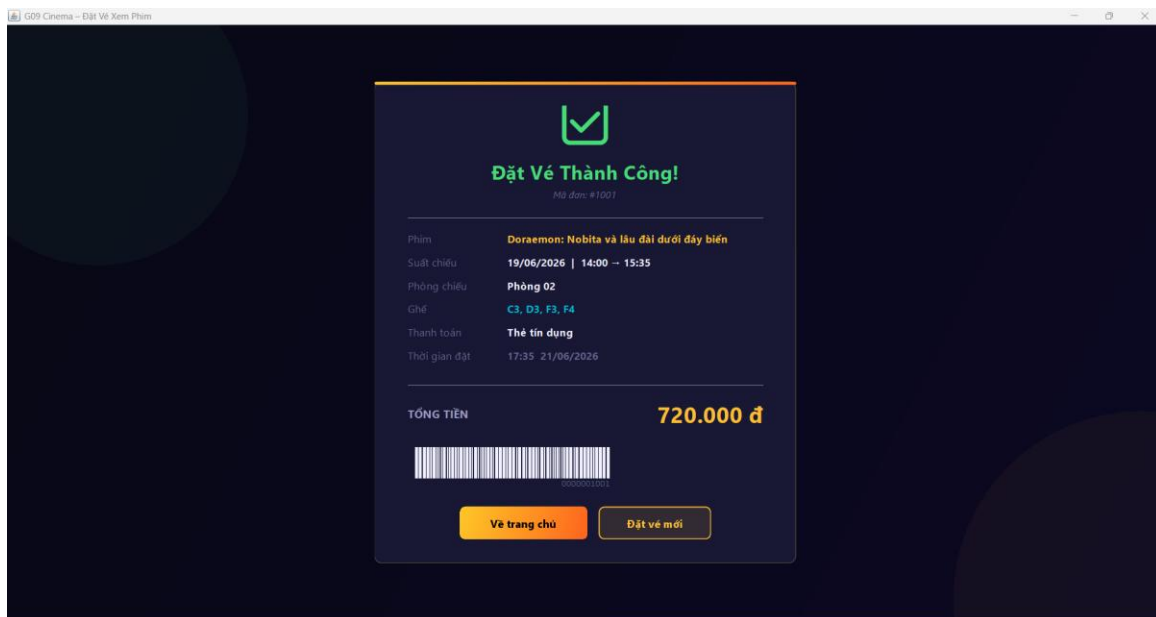
(Thông tin đặt vé)

Ở giao diện cuối cùng này, có thể xem **lại thông tin ghế ngồi**, chọn mua thêm **đồ ăn & nước uống** và chọn phương thức thanh toán.



(Ảnh minh họa)

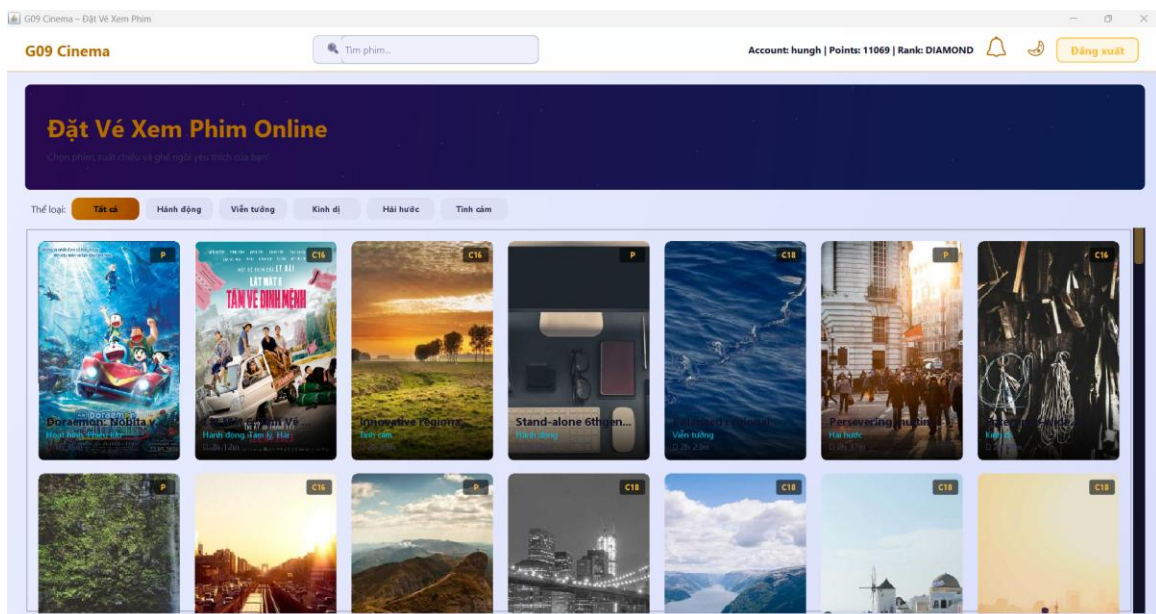
Sau khi đã xác nhận xong, bấm nút **Xác nhận đặt vé**



(Đặt vé thành công!)

Sau khi đặt vé thành công, **thông tin vé** cùng **mã xác nhận** sẽ được hiển thị.

## 5. Chế độ sáng tác tương tác với user



(Ảnh minh họa)

## VIII. Đánh giá hệ thống

### 8.1. Ưu điểm

#### 8.1.1. Tính chuẩn hóa cao (Normalization đạt mức 3NF)

Cấu trúc bảng triệt tiêu được sự dư thừa dữ liệu. Các thông tin có thể sinh ra nhiều giá trị (như số điện thoại, email, diễn viên) đều được tách thành các bảng riêng (Staff\_Contact, Manager\_Contact, Cast\_Member). Điều này giúp CSDL không bị phình to vô ích và tránh các dị thường khi cập nhật.

- **Đạt chuẩn 1NF (Tính nguyên tố):** Toàn bộ các cột trong hệ thống đều chỉ chứa giá trị nguyên tố, không có bất kỳ cột đa trị hay mảng dữ liệu nào. Ví dụ điển hình nhất là thay vì nhồi nhét nhiều số điện thoại hay danh sách diễn viên vào một cột, thiết kế đã tách hẳn ra thành các bảng riêng biệt như Customer\_Phone, Manager\_Contact, và Cast\_Member.
- **Đạt chuẩn 2NF (Xóa bỏ phụ thuộc cục bộ):** Hệ thống không tồn tại sự phụ thuộc một phần trong các bảng có khóa chính ghép, nói cách khác **tất cả thuộc tính không khóa đều phụ thuộc hàm đầy đủ vào khóa chính**. Lấy ví dụ ở bảng Bill\_Detail với khóa chính ghép là (billid, productid), thuộc tính không khóa duy nhất là quantity (số lượng) phụ thuộc hoàn toàn vào *cả hai* mã hóa đơn và mã sản phẩm cộng lại, chứ không thể chỉ suy ra từ một trong hai mã.
- **Đạt chuẩn 3NF (Xóa bỏ phụ thuộc bắc cầu):** Mọi thuộc tính không khóa (Non-prime attribute) đều phụ thuộc trực tiếp và duy nhất vào Khóa chính (Primary Key). **Không có tình trạng cột này suy ra cột kia trong cùng một bảng (mọi thuộc tính không khóa đều không phụ thuộc bắc cầu vào khóa chính)**. Lấy ví dụ ở bảng Staff, các thông tin như role, salary, address đều phụ thuộc hoàn toàn vào staffid. Không có trường hợp lưu thêm cột trung gian nào để gây dị thường dữ liệu.

### **8.1.2. Phân chia Module logic chặt chẽ**

Hệ thống chia rõ 6 cụm nghiệp vụ độc lập nhưng có tính liên kết: Khách hàng, Nhân sự, Phim/Suất chiếu, Hóa đơn/Vé, Sản phẩm F&B, và Đối tác. Việc này giúp mở rộng hệ thống (Scale) sau này rất dễ dàng, ví dụ nếu muốn nâng cấp phân hệ F&B thì không làm ảnh hưởng đến phân hệ Bán vé.

### **8.1.3. Bảo vệ tính toàn vẹn**

Mạng lưới Khóa ngoại (Foreign Key) được thiết lập đầy đủ và chính xác giữa các bảng, đảm bảo không có tình trạng vé không thuộc hóa đơn nào hay suất chiếu không có phòng chứa.

## **8.2. Nhược điểm**

### **8.2.1. Chuẩn hóa quá mức gây áp lực truy vấn**

Việc tách liên lạc thành Staff\_Contact, Manager\_Contact, Cast\_Member mang tính hàn lâm cao, nhưng thực tế làm giảm hiệu năng trầm trọng. Để in thông tin truy xuất của một nhân viên, hệ thống bắt buộc phải thực hiện phép nối (JOIN), gây lãng phí tài nguyên cho những thông tin lẽ ra có thể gộp chung vào bảng gốc.

### **8.2.2. Thiếu nền tảng phát triển ứng dụng Mobile**

Hiện tại, dự án mới chỉ tập trung hoàn thiện tầng lõi cơ sở dữ liệu (Database Layer) và hướng đến các nghiệp vụ quản lý nội bộ (Desktop App). Tuy nhiên, đối với mô hình kinh doanh rạp chiếu phim hiện đại, hơn 80% giao dịch đặt vé và tương tác của khách hàng (Customer) diễn ra trên điện thoại thông minh.

Việc chưa có ứng dụng Mobile (trên iOS/Android) khiến hệ thống bỏ ngỏ trải nghiệm người dùng (UX). Rạp không thể triển khai các tính năng cốt lõi và tiện lợi nhất hiện nay như: đặt vé/chọn ghế nhanh trên app, lưu mã vé điện tử (QR Code) để quét tại cổng kiểm soát, tích điểm thành viên (Membership) tự động, thanh toán qua ví điện tử (Momo/ZaloPay), hay gửi thông báo đẩy (Push Notifications) nhắc nhở lịch chiếu phim.

## 8.3. Những khó khăn trong quá trình phát triển hệ thống & giải pháp

### 8.3.1. Xung đột dữ liệu khi "Tranh cướp vé"

- **Vấn đề:** Trong những dịp bom tấn ra rạp, nhiều khách hàng cùng lúc chọn một ghế (seatid) trong cùng một suất chiếu (movieshowid). Nếu CSDL không có cơ chế khóa, cả 2 người có thể cùng thanh toán thành công cho 1 chiếc ghế.
- **Cách khắc phục:**
  - Tạo thêm ràng buộc duy nhất (Unique Constraint) trên bảng Ticket: UNIQUE (movieshowid, seatid).

### 8.3.2. Tắc nghẽn hiệu năng khi xuất báo cáo tổng hợp

- **Vấn đề:** Thiết kế 3NF chia nhỏ quá nhiều bảng. Nếu Manager yêu cầu xuất báo cáo "Doanh thu tổng hợp theo từng phim, từng rạp, kèm số vé và số bắp & nước bán ra", hệ thống sẽ phải JOIN từ 6 đến 8 bảng cùng lúc, gây nghẽn cổ chai toàn bộ Server.
- **Cách khắc phục:**
  - Sử dụng **Materialized Views** (Khung nhìn thực thể hóa) để CSDL tự động tính toán, gom nhóm sẵn doanh thu vào thời gian trống và lưu thành một bảng tĩnh. Khi được yêu cầu, báo cáo chỉ cần SELECT từ view này với tốc độ phản hồi tính bằng mili-giây.

### 8.3.3. Kiểm soát logic độ tuổi và giá vé

- **Vấn đề:** Hiện tại, giá vé đang được lưu cứng ở bảng Ticket, nhưng giá vé thực tế phụ thuộc vào loại ghế (VIP/Thường), ngày chiếu (Cuối tuần/Ngày thường) và đối tượng (Học sinh/Người lớn). Việc để cột ticketprice độc lập rất dễ bị nhân viên nhập sai giá. Ngoài ra, độ tuổi khách (dob) chưa được đối chiếu tự động với nhân phim (agerating).

- **Cách khắc phục:**

- Viết các **Triggers** trước khi thực hiện thao tác INSERT vào bảng Ticket. Trigger sẽ làm 2 việc:
  - Một là tính tuổi khách hàng so với nhãn phim (Từ chối xuất vé nếu dưới tuổi)
  - Hai là tự động tra bảng "Bảng giá" để điền ticketprice chính xác dựa trên seattype và showdate, không cho phép nhập tay trực tiếp.

**--- KẾT THÚC ---**